



第三章 数据结构

朱容波

Email: rongbozhu@126.com

Office: 一综B323

华中农业大学信息学院

本章内容



- 基本数据结构:
 - 链表(循环链表, 双向链表, 循环双向链表)
 - 堆栈和队列
 - 图(平面图)
 - 树, 根树, 二叉树, 二叉搜索树

华中农业大学信息学院

例1 书目自动检索系统

线性表



书目文件

001	高等数学	樊映川	S01
002	理论力学	罗远祥	L01
003	高等数学	华罗庚	S01
004	线性代数	栾汝书	S02
.....	书名:....		

按书名

高等数学	001, 003, ...
理论力学	002, ...
线性代数	004, ...
.....	

作者名:
分类号:

按作者名

樊映川	001, ...
华罗庚	002, ...
栾汝书	004, ...
.....	

按分类号

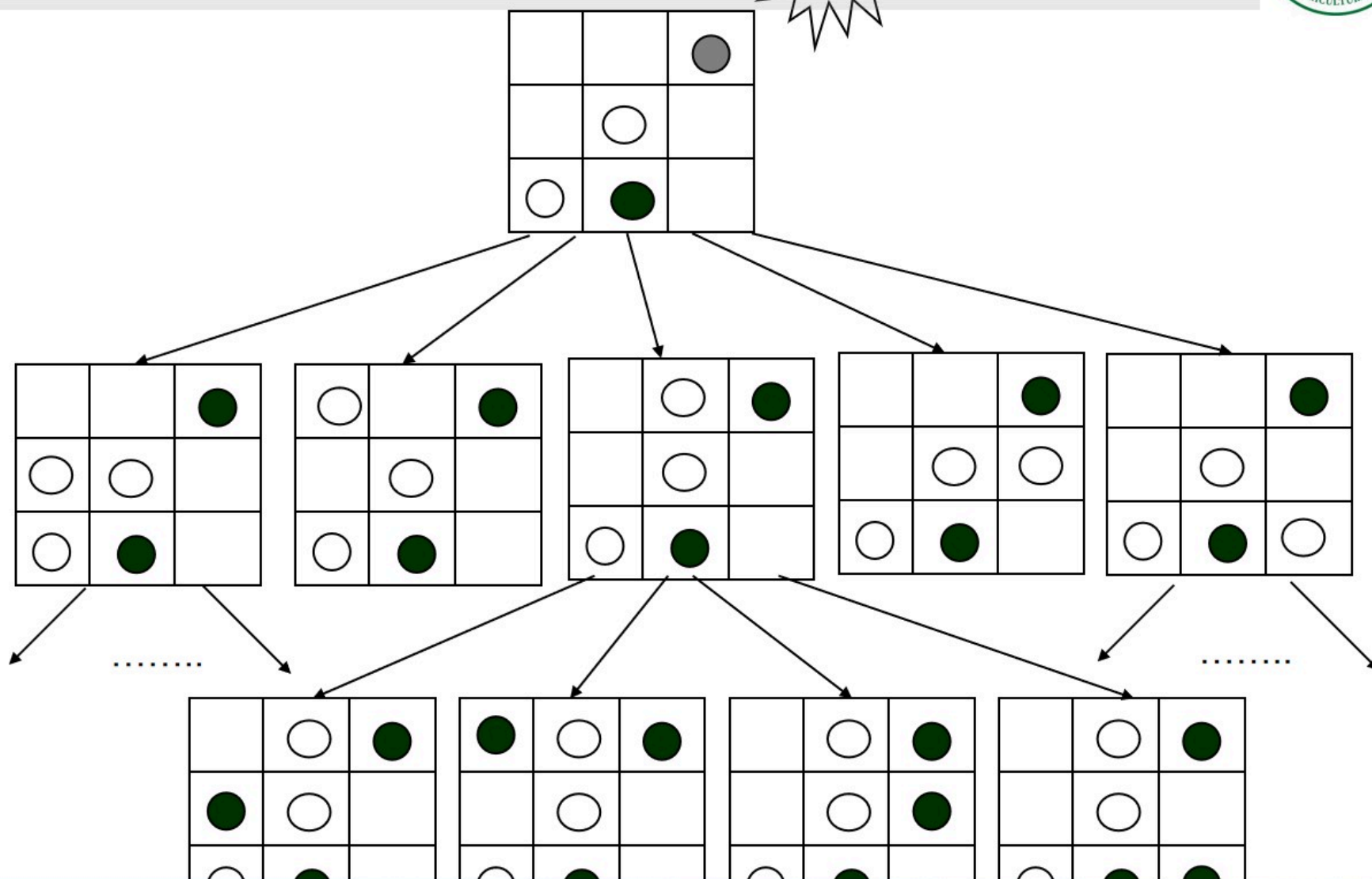
L	002, ...
S	001, 003, ...
.....	

索引表

华中农业大学信息学院

例2 人机对弈问题

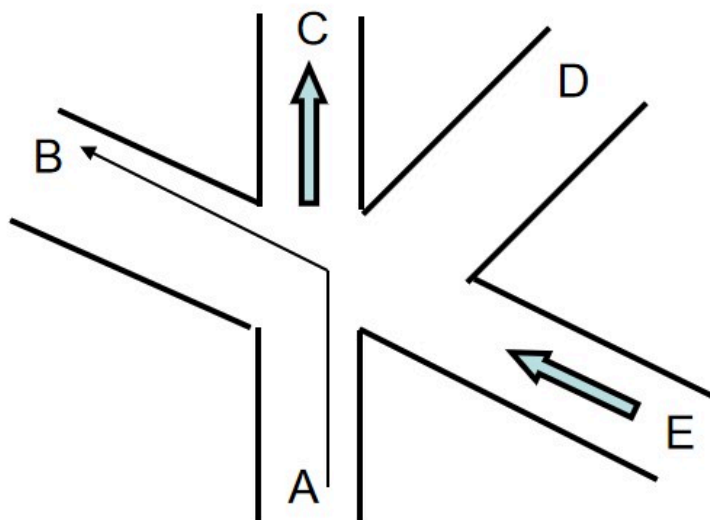
树



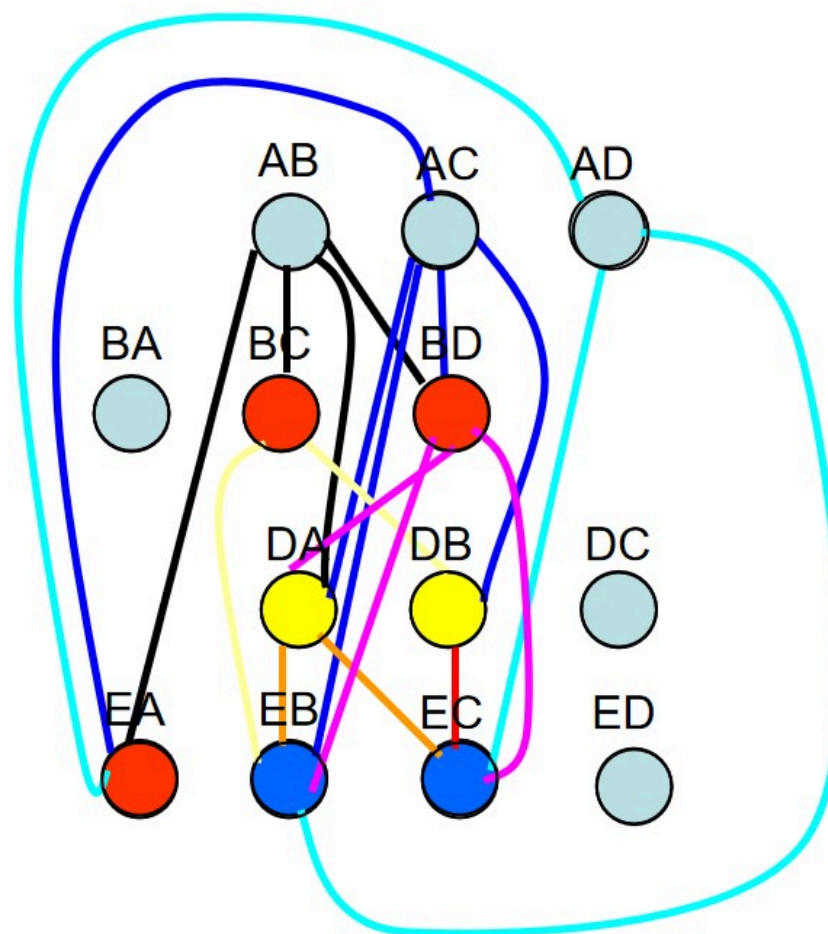
华中农业大学信息学院

例3 多叉路口交通灯管理问题

图



顶点表示通路，通路之间的连线表示两条通路不能同时通行。



华中农业大学信息学院

基本概念和术语



数据(Data):是对客观事物的符号表示。在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称。

数据元素(Data Element):数据的基本单位。在计算机程序中通常作为一个整体进行考虑和处理。一个数据元素可由若干个数据项组成。数据项是数据的不可分割的最小单位。

数据元素			
001	高等数学	樊映川	S01
002	理论力学	罗远祥	L01
003	高等数学	华罗庚	S01
004	线性代数	栾汝书	S02
.....			



数据对象(Data Object): 是性质相同的数据元素的集合。
是数据的一个子集。

数据结构(Data Structure): 是相互之间存在一种或多种特定关系的数据元素的集合。

数据结构包括逻辑结构、物理结构和施加在数据上的运算

例如:

3214,6587,9345 — **a1(3214),a2(6587),a3(9345)**

则在数据元素 a1、a2 和 a3 之间存在着“次序”关系

<a1,a2>、<a2,a3>

3214,	6587,	9345	≠	6587,	3214,	9345
a1	a2	a3		a2	a1	a3

5. 逻辑结构

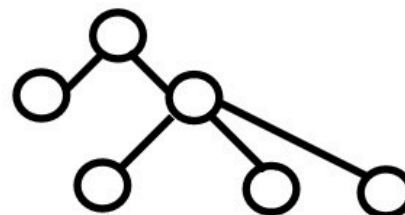
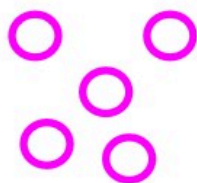
是对数据元素之间逻辑关系的描述。根据数据元素之间的逻辑结构可将数据结构分为四类：

集合 —— 数据元素除了同属于一种类型外，别无其它关系。

线性结构 —— 数据元素之间存在一对一的关系。如线性表、栈、队列。

树形结构 —— 数据元素之间存在一对多的关系。如树。

图状结构（或网状结构） —— 数据元素之间存在多对多的关系，如图。



华中农业大学信息学院



数据元素之间的关系在计算机中有两种不同的表示方法：

顺序映像和非顺序映像。分别对应两种存储结构：

- 顺序存储结构：借助元素在存储器中的相对位置来表示数据元素间的逻辑关系。
- 链式存储结构：借助指示元素存储地址的指针表示数据元素间的逻辑关系。

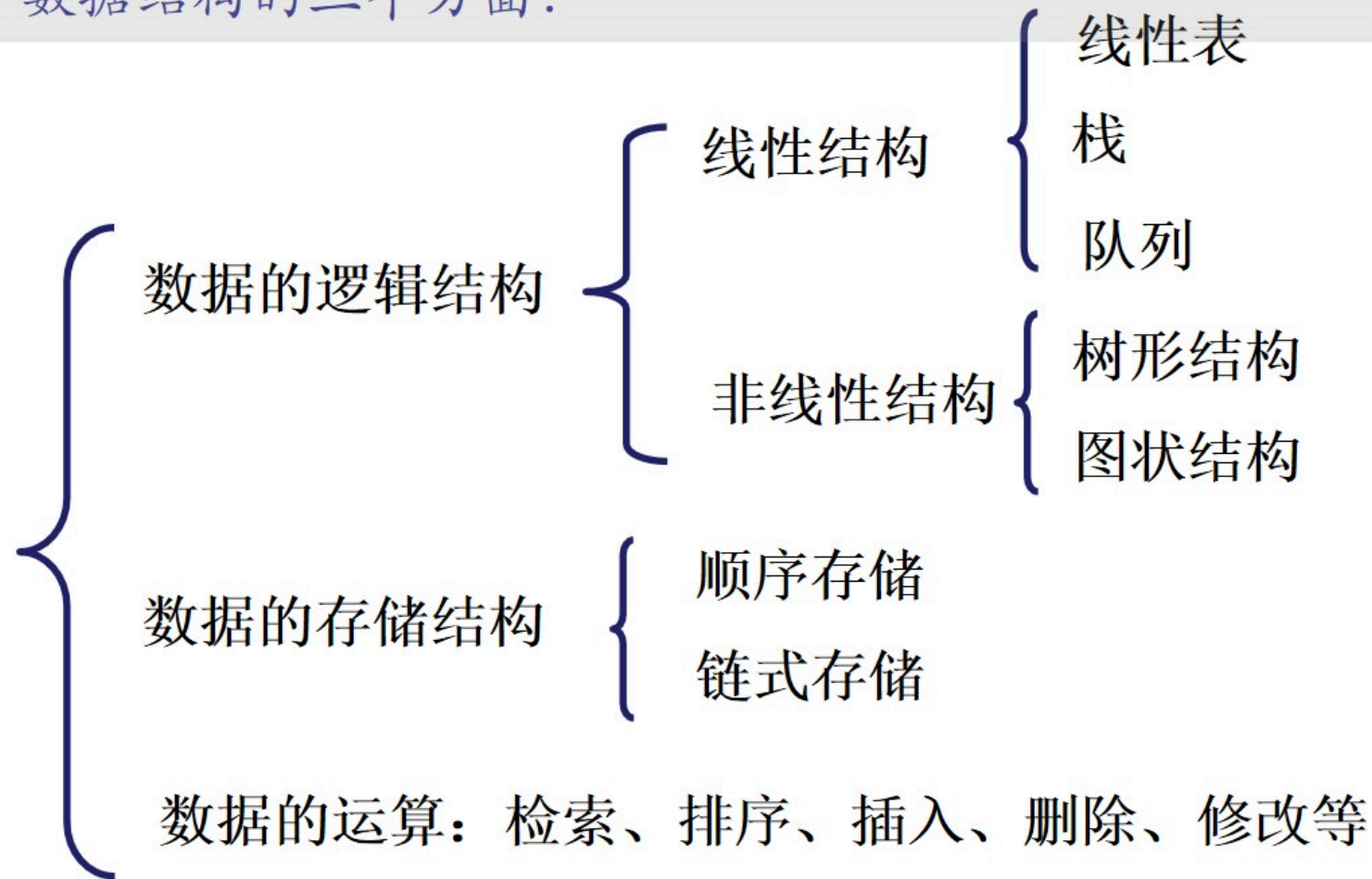
数据的逻辑结构与存储结构密切相关：

算法设计 $\longrightarrow$ 逻辑结构

算法实现 $\longrightarrow$ 存储结构



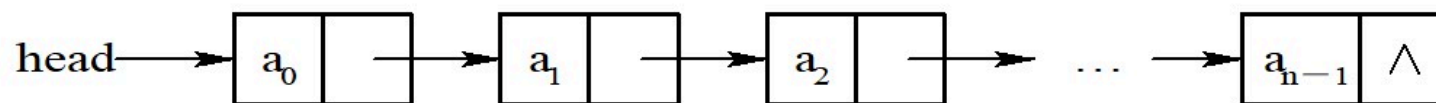
数据结构的三个方面：





3.2 链表

- **链表**由有限的元素或节点序列组成，节点包含信息和到另一个节点的链。
- 如果节点x指向节点y，那么x称为y的**前驱节点**，y称为x的**后续节点**。
- 指向第一个元素的链称为**表头**

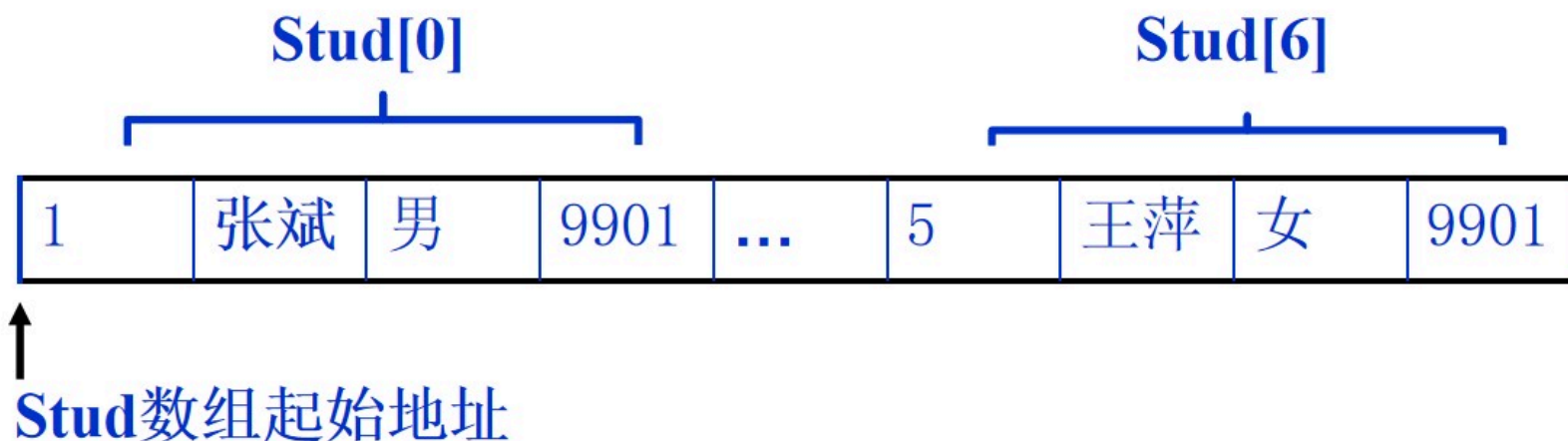




存放学生表的结构体数组**Stud**定义为:

```
struct
{
    int no;           /*存储学号*/
    char name[8];     /*存储姓名*/
    char sex[2];      /*存储性别*/
    char class[4];    /*存储班号*/
} Stud[7]={1,“张斌”,“男”,“9901”,...,
          {5,"王萍","女","9901"}};
```


结构体数组**Stud**各元素在内存中顺序存放,即第 **i** ($1 \leq i \leq 6$)个学生对应的元素**Stud[i]**存放在第 **$i+1$** 个学生对应的元素**Stud[i+1]**之前,**Stud[i+1]**正好在**Stud[i]**之后。

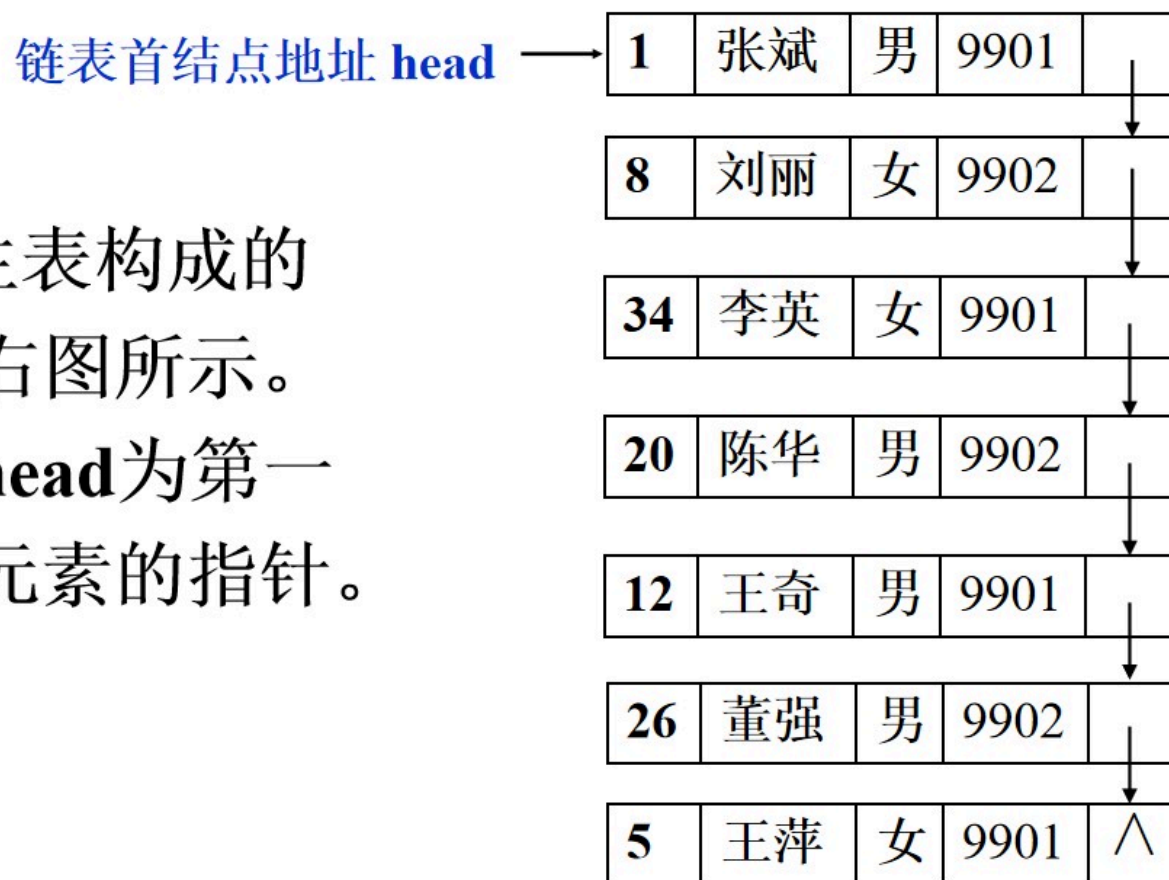




存放学生表的链表的结点类型**StudType**定义为:

```
typedef struct studnode  
{    int no;                /*存储学号*/  
        char name[8];        /*存储姓名*/  
        char sex[2];         /*存储性别*/  
        char class[4];       /*存储班号*/  
        struct studnode *next; /*存储指向下一个学生的指针*/  
} StudType;
```

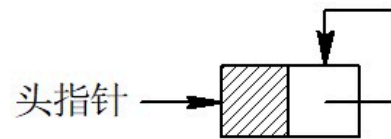
学生表构成的
链表如右图所示。
其中的**head**为第一
个数据元素的指针。



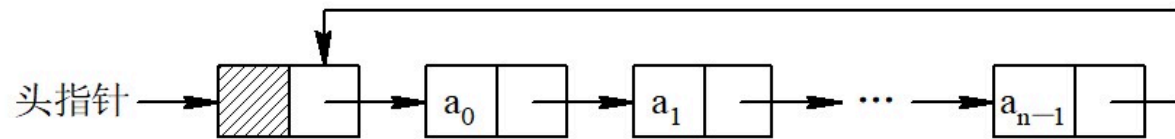


链表

- 如果链表中存在由最后一个元素到第一个元素的链，那么这种链表称为**循环链表**
- 如果在一个链表中每一个节点（第一个节点可能除外）也指向它的前驱节点，那么这个链表称为**双向链表**；
- 如果第一个和最后一个节点也被一对链连接起来，这就是一个**循环双向链表**。



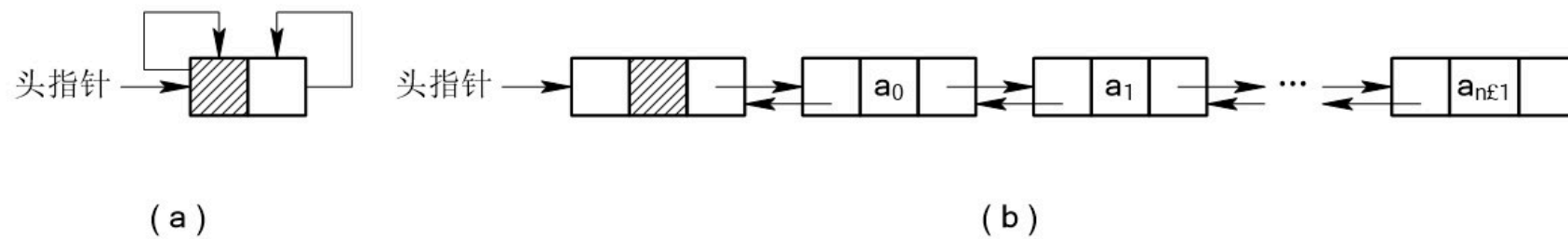
(a)



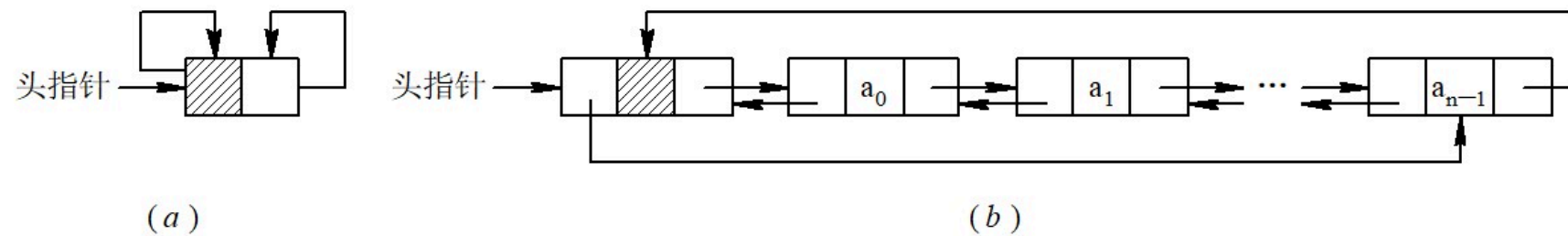
(b)

带头结点的循环链表

(a)空链； (b)非空链



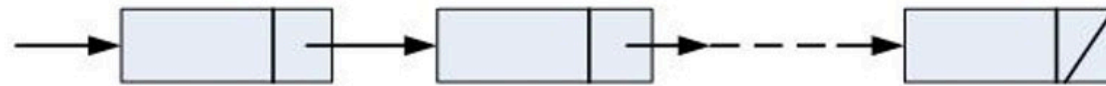
带头结点的双向链表
(a)空链； (b)非空链



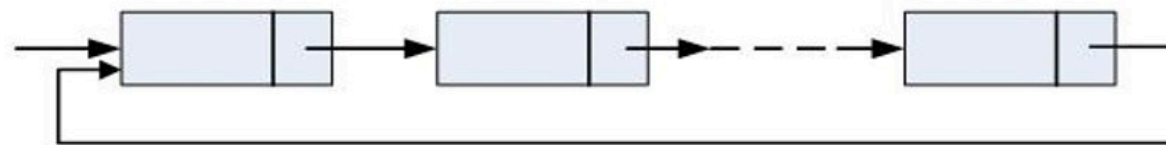
带头结点的循环双向链表

(a)空链； (b)非空链

链表



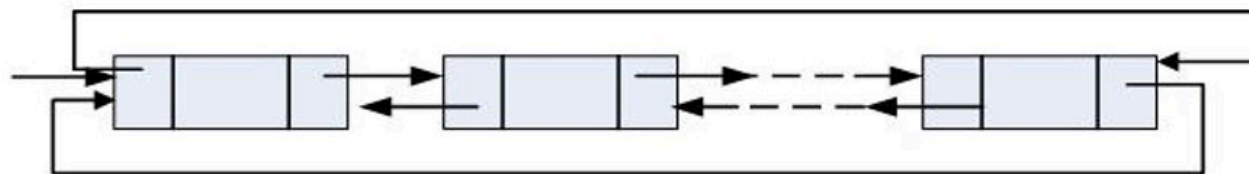
Linked list



Circular linked list



Doubly linked list



Circular doubly linked list



2个主要操作

- 插入
- 删除
- 与数组不同，在链表中插入和删除元素只花费**固定的时间**。

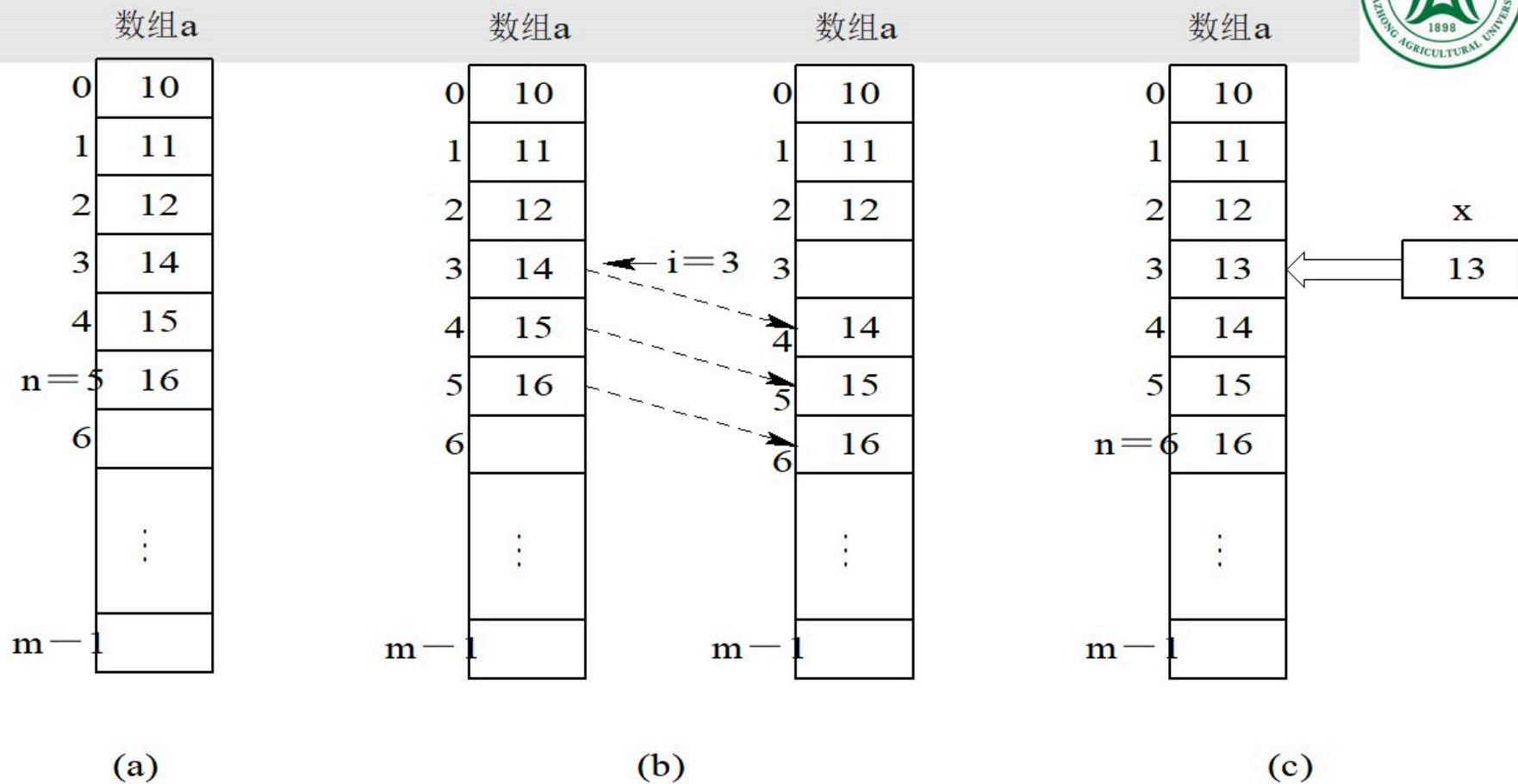


图 数组存储下线性表的插入过程
(a) 插入前; (b) 后移; (c) 插入

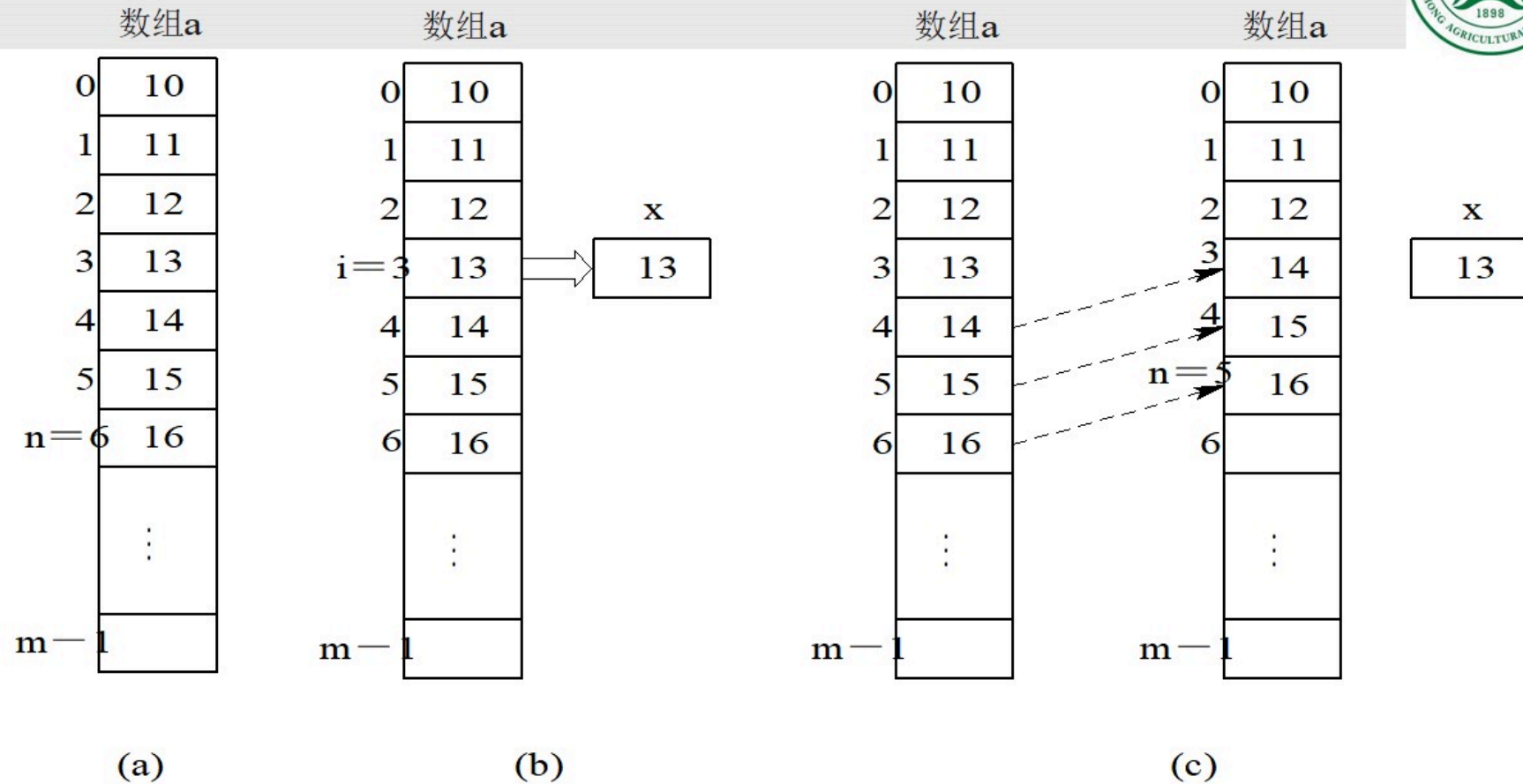
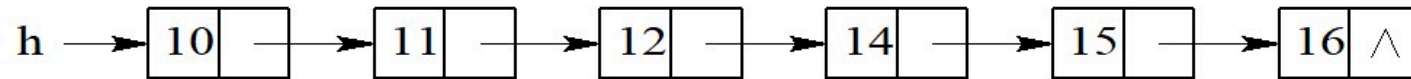


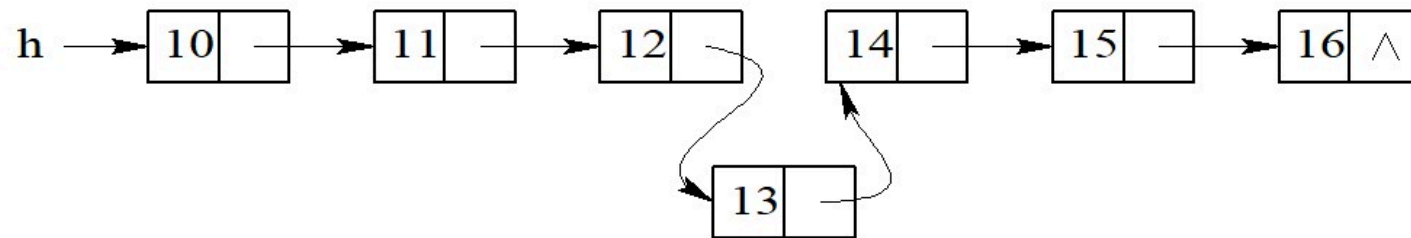
图 数组存储下线性表的删除过程
(a) 删除前; (b) 保存; (c) 前移



(a)



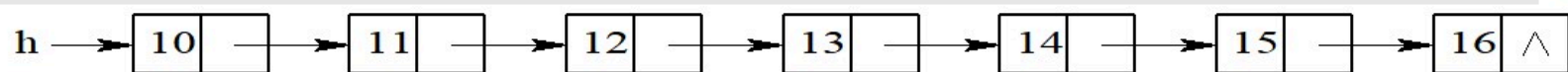
(b)



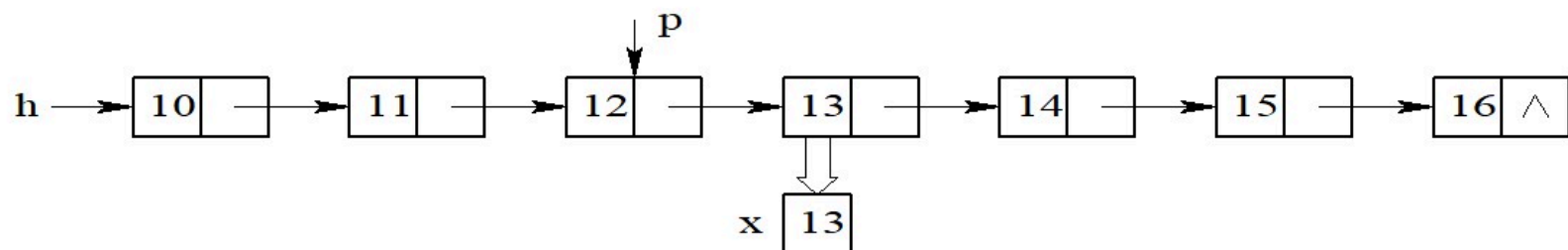
(c)

图 链式存储下线性表的插入过程

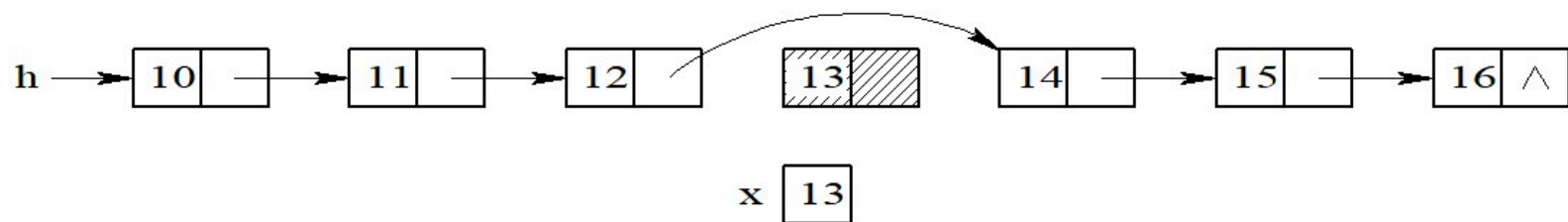
(a) 插入前; (b) 新结点赋值; (c) 链入



(a)



(b)



(c)

图 链式存储下线性表的删除过程

(a) 删除前; (b) 保存; (c) 删除



3.2.1 堆栈和队列

- 对链表的访问加入一些限制，就可以得到两种基础性的数据结构：堆栈和队列
- 堆栈是一种只允许在称为栈定的一端进行插入和删除运算的链表，也可以在数组中实现这些运算。
- 两种基本操作：
 - 如果S是一个堆栈，如果x是与S中元素类型相同的一个元素
 - **Push运算**: $\text{push}(S, x)$ 把x加到S中，并改变堆栈的顶部，使它指向x。
 - **Pop运算**: 返回栈顶元素，并将它从堆栈中永久地移去。

华中农业大学信息学院



3.2.1 堆栈性质

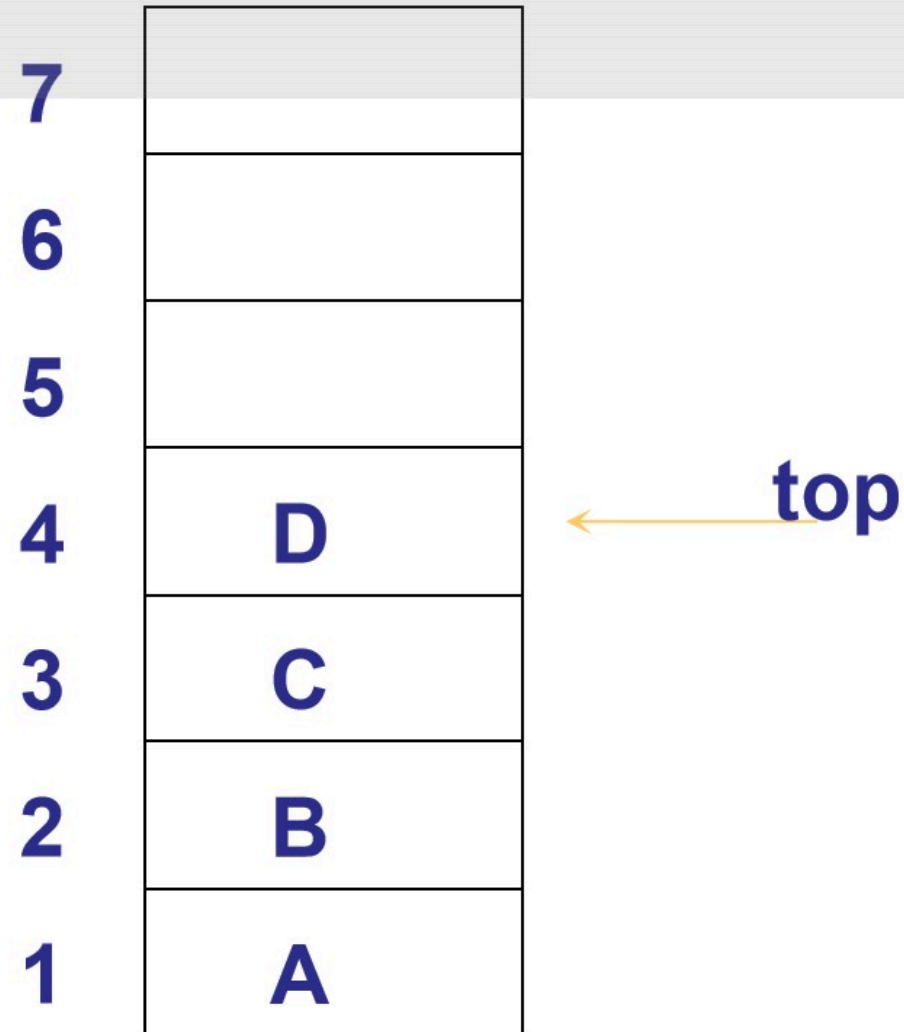
- 堆栈也简称为**栈**，是限定在表的一端进行插入或删除操作的线性表。
- 进行插入或删除操作的一段称为**栈顶 (top)**，另一端称为**栈底 (bottom)**。
- 插入元素又称为**入栈 (push)**，删除元素操作称为**出栈 (pop)**。
- 不含元素的栈称为**空栈**。
- 堆栈元素的插入和删除只在栈顶进行，总是后进去的元素先出来，所以堆栈又称为**后进先出线性表**或 LIFO (Last-In-First-Out) 表。



3.2.1 堆栈性质

- 堆栈的最简单的表示方法是采用一维数组，为形象起见，一般在图中将堆栈画成竖直的。
- 设数组名为STACK，其下标的下界为1，上界为n。
- 一般需用一个变量top记录当前栈顶的下标值，**top也叫做栈指针。**

- 本例中 $\text{top}=4$



STACK



堆栈处理算术表达式

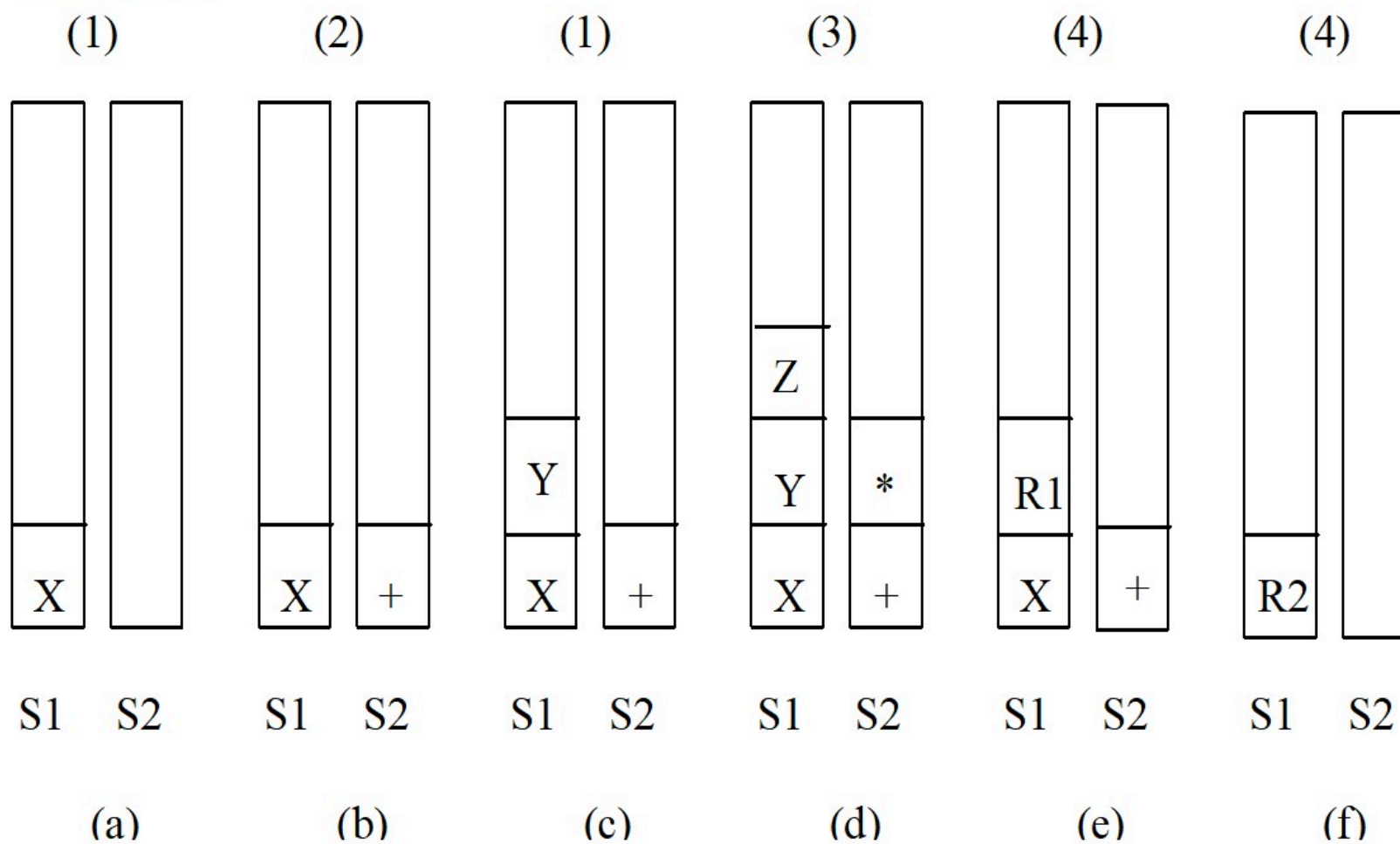
- 下面用一个简单的例子说明编译系统在处理算术表达式时，是如何应用堆栈这种数据结构的。
- 假定表达式的运算数都是使用单个字母表示的，式中无括号且只有加、减、乘、除4种运算，而没有更复杂的运算，例如表达式 $X+Y*Z$ 。



- 使用S1和S2两个堆栈，S1用于存储运算数，S2用于存储运算符。
- 编译系统处理时，将表达式从左向右逐个扫视一遍，并根据不同情况按以下原则处理：
 - 1) 若是运算数，则将其压入S1栈；
 - 2) 若是运算符且S2栈是空栈则将其压入 S2栈；
 - 3) 若是运算符且S2栈为非空栈，且此运算符的级别高于S2栈顶运算符的级别，则将此运算符压入S2栈；
 - 4) 凡不属于上面三条的情况，则将S2的栈顶运算符与S1栈最上面的两个运算数出栈进行运算，并将运算结果压入S1栈。



- 图中每一步上面括号中的数字表示该步是按哪一条原则处理的。



华中农业大学信息学院



队列

- 队列是这样的一种链表：仅允许在称为队列尾部的链表一端进行插入运算，而只允许在称为队列头部的另一端进行所有的删除运算。
- 和堆栈一样，队列也可以在数组中实现这些运算。除了push运算把一个元素加入了队列尾部之外，队列支持的其他运算和堆栈一样。



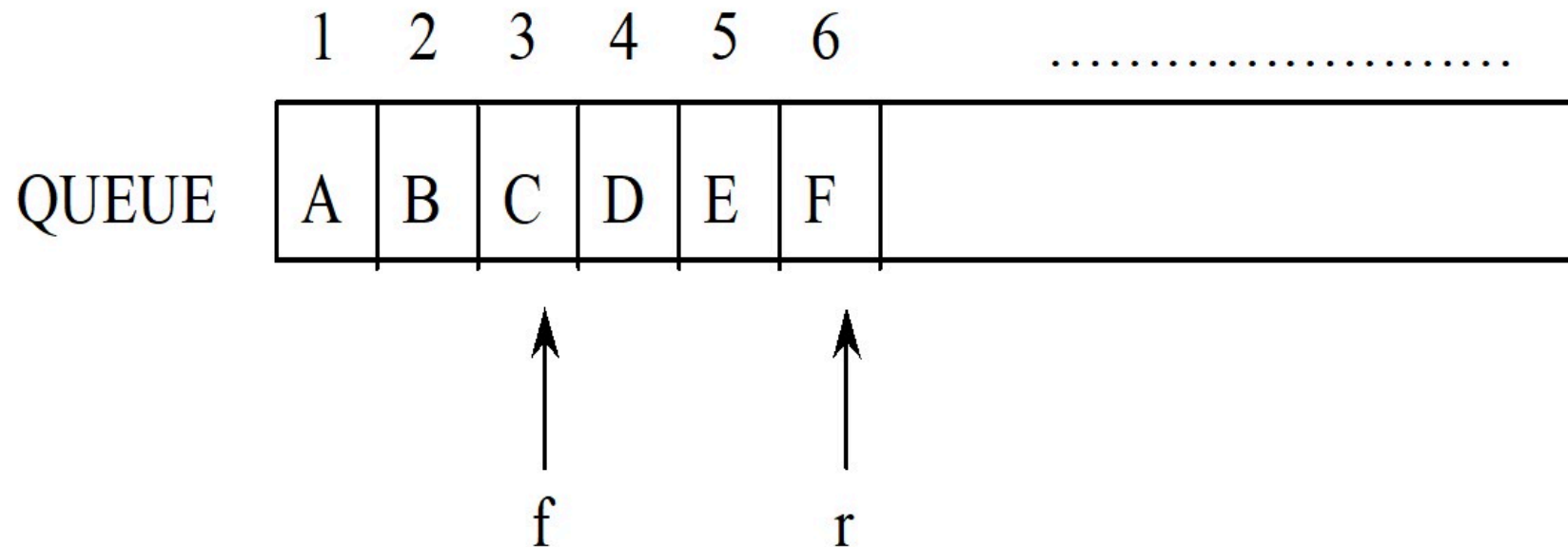
队列

- 队列是一种运算受限制的线性表，元素的添加在表的一端进行，而元素的删除在表的另一端进行。
- 允许添加元素的一端称为**队尾 (Rear)**；允许删除元素的一端称为**队头 (Front)**。
- 向队列添加元素称为**入队**，从队列中删除元素称为**出队**。
- 新入队的元素只能添加在队尾，出队的元素只能是删除队头的元素，队列的特点是先进入队列的元素先出队，所以队列也称作**先进先出表**或FIFO (First-In-First-Out) 表。



队列

- 假定有A~F 6个元素先后进入队列，但A、B两个元素已陆续出队了，故队尾指针 $r=6$ ，而队头指针 $f=3$ 。



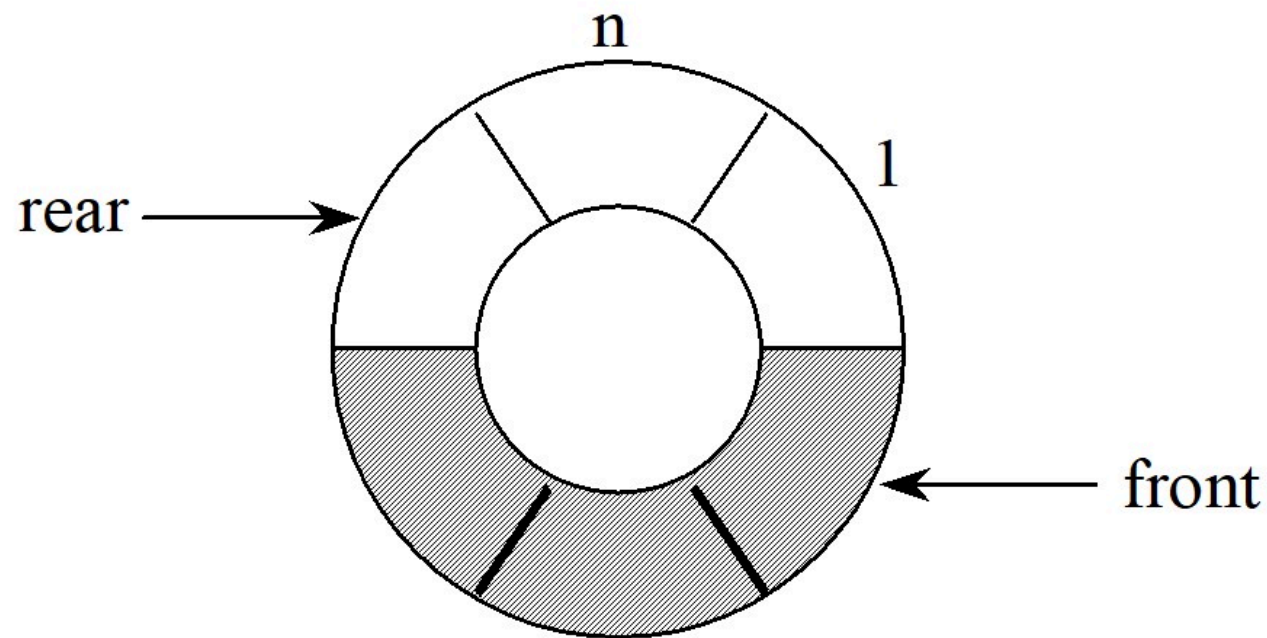


队列存在的问题

- 由于队列的入队操作是在两端进行的，随着元素的不断插入，删除，两端都向后移动，队列会很快移动到数组末端造成溢出，而前面的单元无法利用。
- 解决办法：
 - 1) 每次删除一个元素后，将整个队列向前移动一个单元，保持队列头总固定在数组的第一个单元。
 - 2) 将所用的数组想象成是头尾相接的圆环，当队列的尾端到达数组的末端（第 n 个单元）时，如果再插入元素可继续使队列向数组的前端（第1个单元）延长，此队列称为循环队列。

循环队列

- 图中阴影部分为队列中元素。
- 如何判断一个循环队列是满还是空？





判断循环队列是否满或空

- 满：队尾经过一个循环而到达队首的前一个单元时，这种情况下如果再插入新的元素时，新元素就要把原队头的元素覆盖，因此，当 $r=f$ 时，插入新的元素会造成队列首尾重叠；
- 空：在队列进行删除运算时，当 $f=r$ 时表明删除的是队列的最后一个元素，删除这个元素后，队列就变成空队列。



3.4 树

- 一个自由树（或简称树）是**不包含回路**的连通无向图。一个森林是顶点不相交的树的集合，即它们没有公共的顶点。

- 定理 3.1



如果 T 是有 n 个顶点的树，那么

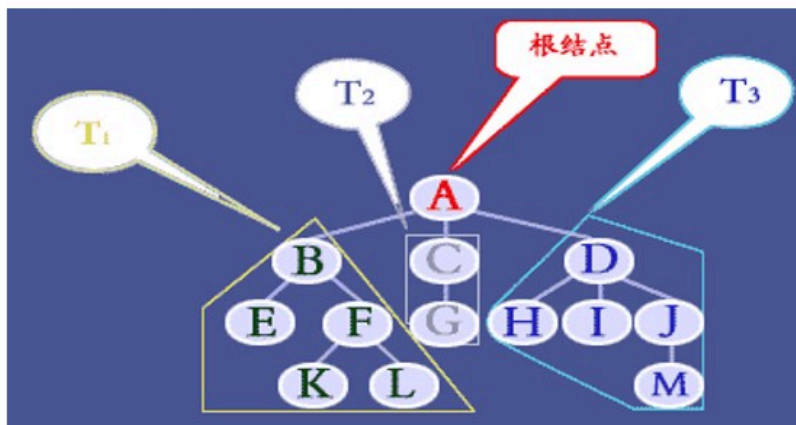
- T 中任意两个顶点有唯一的一条路径连通；
- T 恰好有 $n-1$ 条边；
- 在 T 中加上一条边将构成一个回路。



- 树是一类重要的非线性结构。
- 树是结点之间有分支，并且具有层次关系的结构，它非常类似于自然界中的树。
- 树在客观世界中是大量存在的，例如家谱、行政组织机构都可用树形象地表示。
- 树在计算机领域中也有着广泛的应用，例如
 - 在编译程序中，用树来表示源程序的语法结构；
 - 在数据库系统中，可用树来组织信息；
 - 在分析算法的行为时，可用树来描述其执行过程。等等。

3.5 根树

- 根树 T 是一棵带有特殊顶点 r 的树，顶点 r 称为 T 的**根**。
- 在 T 中，如果顶点 v_i 在从根到 v_j 的路径上，并且和 v_j 相邻，就说 v_i 是 v_j 的**父顶点**，同时， v_j 是 v_i 的**子顶点**。
- 一个顶点的子顶点之间彼此是**兄弟**。没有子顶点的顶点是树根的**叶子**，其他所有的顶点都称为**内部顶点**。
- 在从根顶点到顶点 v 的路径上的顶点 u 是 v 的**祖先**，如果 $u \neq v$ ， u 就是 v 的**真祖先**；在从顶点 v 到叶子的路径上的顶点 w 是 v 的**后代**，如果 $w \neq v$ ， w 就是 v 的**真后代**。





根树

- 子树:
 - 从顶点 v 为根的子树是包括顶点 v 和它的真后代的树
- 深度:
 - 在根树中顶点 v 的深度是从根顶点到 v 的路径的长度。这样，根顶点的深度为0。
- 高度:
 - 顶点 v 的高度定义为从顶点 v 到叶子中最长路径的长度，一棵树的高度是根顶点的高度。



3.5.1 树的遍历

有几种方法可以对一棵根树的顶点进行有系统的遍历和排序，其中最重要的三种是前序，中序和后序遍历。令 T 是一颗树，它的根顶点是 r ， $T_1, T_2, \dots, T_n$ 是子树。

- 前序遍历:

在 T 顶点的前序遍历中，先访问根顶点 r ，然后以前序遍历 T_1 的顶点，之后前序遍历 T_2 的顶点。如此继续，直到前序遍历 T_n 的所有顶点。

- 中序遍历:

在 T 顶点的中序遍历中，先以中序遍历 T_1 的顶点，然后访问根顶点 r ，之后中序遍历 T_2 的顶点。如此继续，直到中序遍历 T_n 的所有顶点。

- 后序遍历:

在 T 顶点的后序遍历中，先以后序遍历 T_1 的顶点，然后后序遍历 T_2 的顶点。如此继续，直到后序遍历 T_n 的所有顶点，最后访问根顶点 r 。



- 1、先序遍历二叉树的操作定义为：
若二叉树为空，则空操作；否则
 - (1) 访问根结点；
 - (2) 先序遍历左子树；
 - (3) 先序遍历右子树。
- 2、中序遍历二叉树的操作定义为：
若二叉树为空，则空操作；否则
 - (1) 中序遍历左子树；
 - (2) 访问根结点；
 - (3) 中序遍历右子树。
- 3、后序遍历二叉树的操作定义为：
若二叉树为空，则空操作；否则
 - (1) 后序遍历左子树；
 - (2) 后序遍历右子树；
 - (3) 访问根结点。

例：如图所示的二叉树表达式

$(a+b*(c-d)-e/f)$

•若先序遍历此二叉树,按访问结点的先后次序将结点排列起来,其先序序列为:

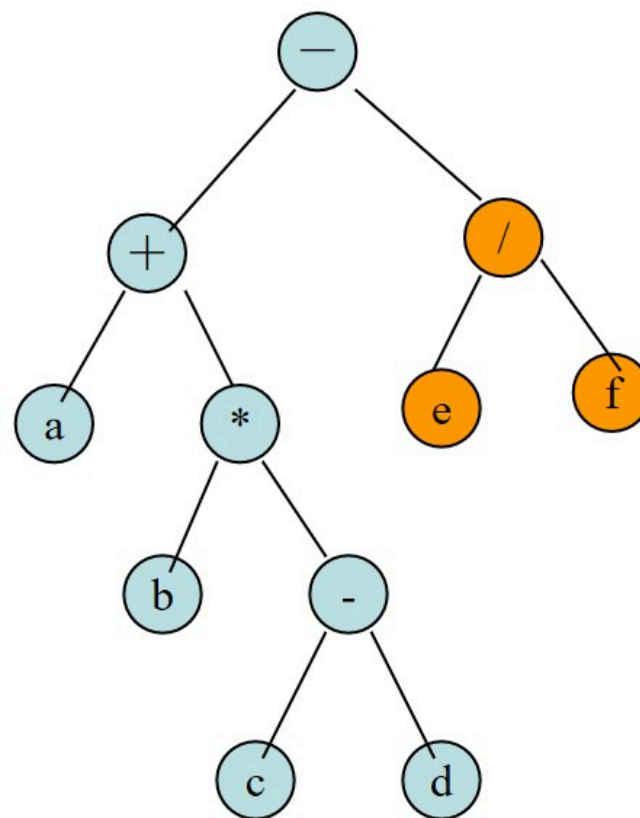
$-+a*b-cd/ef$

•按中序遍历,其中序序列为:

$a+b*c-d-e/f$

•按后序遍历,其后序序列为:

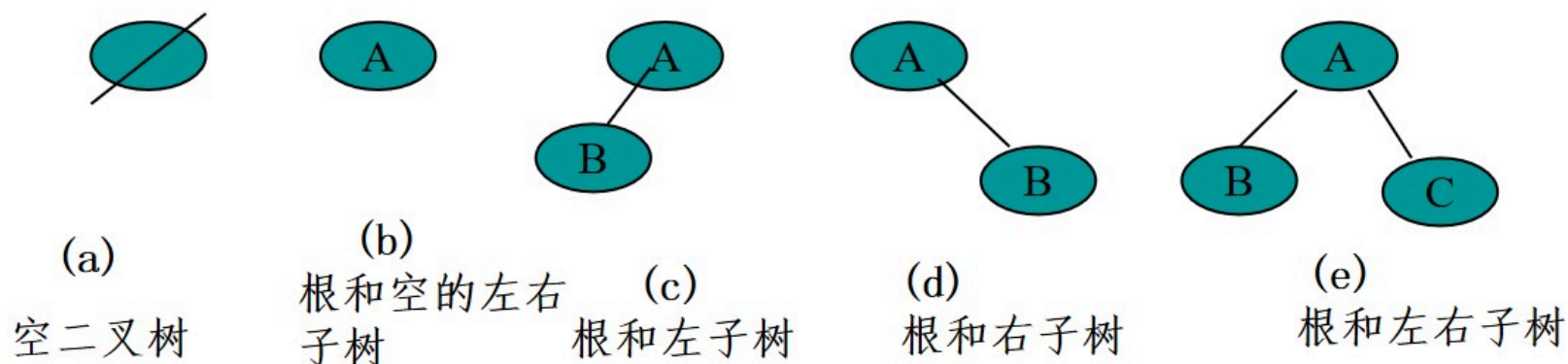
$abcd-*+ef/-$



3.6 二叉树

定义

- 二叉树是节点的一个有限集合，集合或者为空，或者由一个根节点r和称为**左右子树**的两个不相交的二叉树组成。这些子树的根称为r的左右儿子。
- 二叉树与根树之间有两个很重要的不同点：
 - 第一，二叉树可以为空而根树不能为空。
 - 第二，由于二叉树有左右子树的区别，使得图.3.6 中 (A)、(B) 两颗二叉树是不同的，但是如果是根树，他们就不能区分。





二叉树

- 满的和完全的:

- 如果二叉树中每个内部节点都正好有两个儿子, 这样的二叉树称为**满的**; 如果二叉树是满二叉树, 而且所有的叶子有同样的深度 (如在同一层), 那么这种二叉树称为**完全二叉树**。

- 几乎完全的:

- 如果一颗二叉树除了最右边位置上的一个或者几个叶子可能缺少外, 它是丰满的, 我们定义它为几乎完全的二叉树。

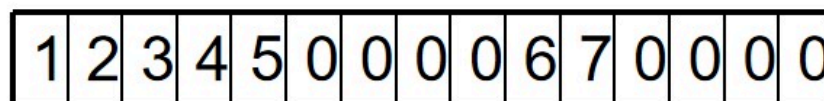
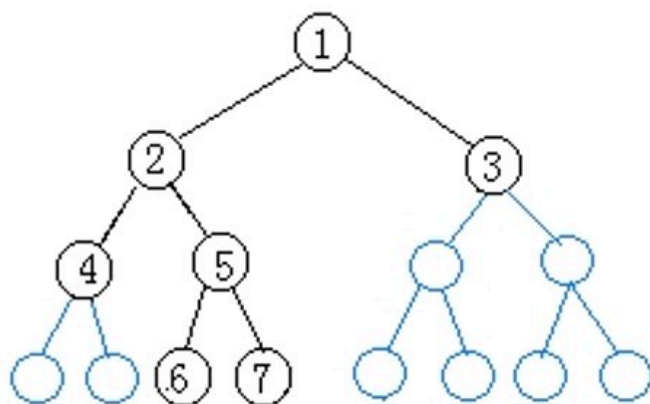


二叉树的存储结构

- 一颗有 n 个顶点的完全（或几乎完全）的二叉树可以用数组 $A[1..n]$ 来有效的表示，数组根据下面的简单关系列出二叉树的顶点：
 - 如果存储在 $A[j]$ 中的顶点有左儿子和右儿子，就把他们分别存储在 $A[2j]$ 和 $A[2j+1]$ 中。
 - 存储在 $A[j]$ 中的顶点的父顶点存储在 $A[\lfloor j/2 \rfloor]$ 中。

二叉树的顺序存储结构

- 二叉树的顺序存储结构用一组连续的存储单元存储二叉树的数据元素。因此，必须把二叉树的所有结点安排成为一个恰当的序列，以使得结点在这个序列中的相互位置能反映出结点之间的逻辑关系，为此使用编号法。即从树根起，自上层至下层，每层自左至右的给所有结点编号。
- ```
#define MAX_TREE_SIZE 100
typedef TElemType SqBiTree[MAX_TREE_SIZE];
SqBiTree bt;
```





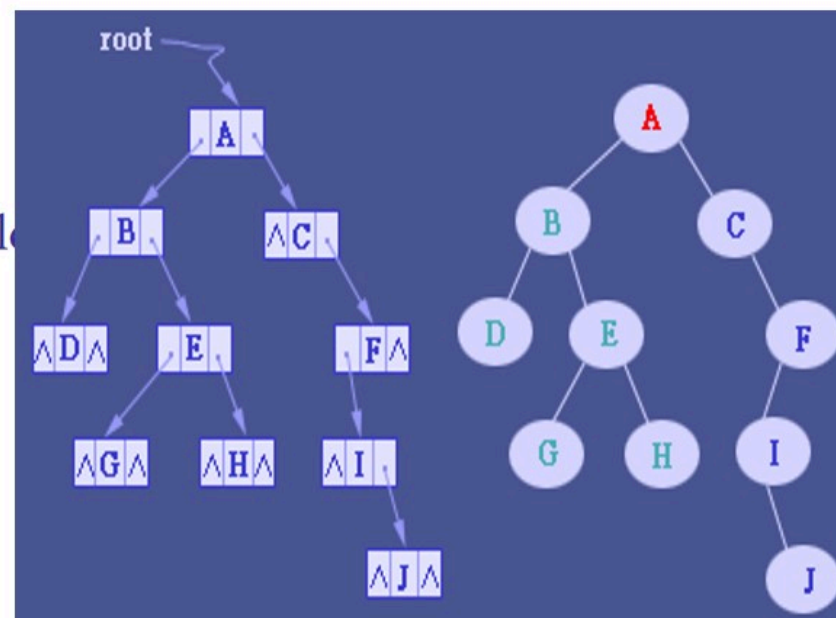
## 二叉树的链式存储结构

- 由二叉树的定义可知，二叉树的结点由一个数据元素和分别指向其左、右子树的两个分支构成，则表示二叉树的链表结点中至少包含三个域：数据域、左指针域及右指针域。

|        |      |        |
|--------|------|--------|
| Lchild | item | Rchild |
|--------|------|--------|

- typedef struct BiTNode{
  - TElemType data;
  - struct BiTNode \*lchild,\*rchild;
 }BiTNode,\*BiTree;

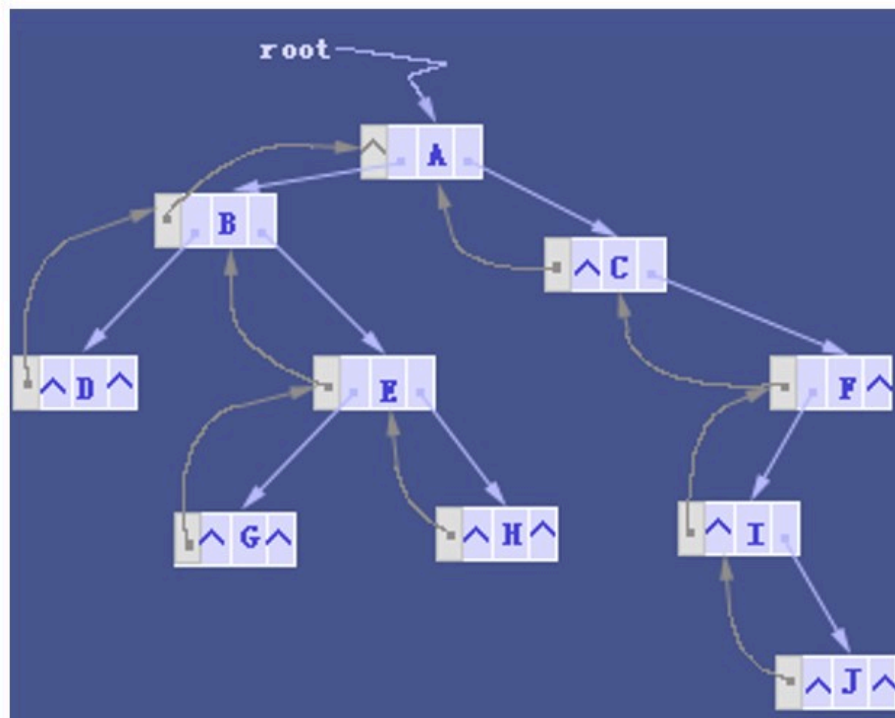
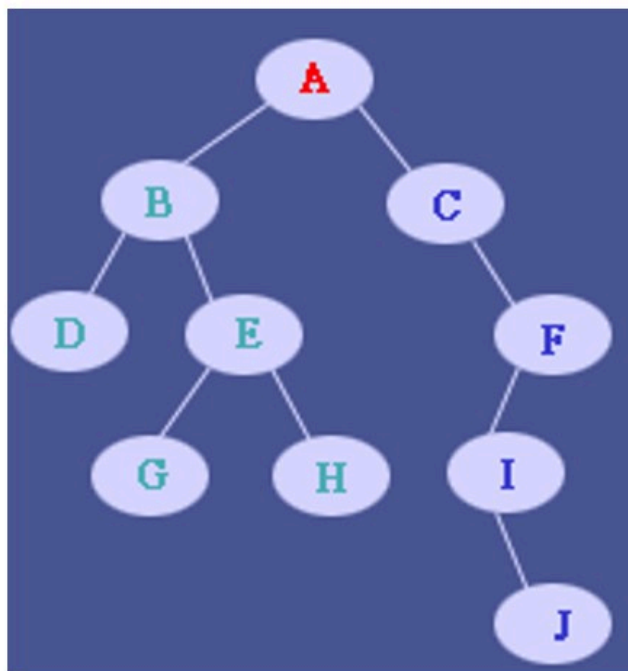
- 二叉链表





- 二叉链表的存储特点是寻找孩子结点容易，双亲比较困难。因此，若需要频繁地寻找双亲，可以给每个结点添加一个指向双亲结点的指针域，其结点结构如下所示。

| lchild | data | parent | rchild |
|--------|------|--------|--------|
|--------|------|--------|--------|





### 3.6.1 二叉树的一些定量特征

- 在下面的观察结论中，列出了二叉树的层、顶点数和高度间的一些有用的关系。
- 观察结论3.1 在二叉树中，第j层的顶点数最多是 $2^j$ 。
- 观察结论3.2 令二叉树T的顶点数是n，高度是k，那么：

$$n \leq \sum_{j=0}^k 2^j = 2^{k+1} - 1.$$

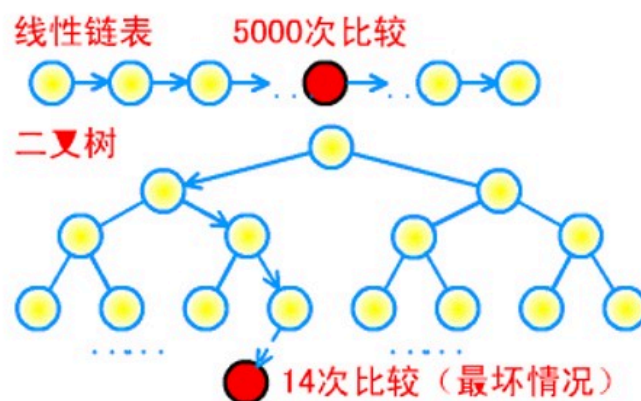
如果T是完全的，等号成立。如果T是几乎完全的，那么有

$$2^k \leq n \leq 2^{k+1} - 1.$$

其他观察结论见 P71-72

## 二叉搜索树

- 事实上，二分查找的过程，隐含地对应一棵二叉树形，但这样的二叉树形仅是逻辑上的，不能进行插入和删除操作。如果用二叉树形来组织表，则我们将会得到较好的动态查找算法。
- 一般来说，二叉树结构可以显著地改善搜索的性能，因为达到一个树的路径最长不超过树深度。







## 二叉搜索树

- 二叉搜索树是顶点用线性集中的元素来标记的一种二叉树，标记的方法是：所有存储在顶点 $v$ 的左子树中的元素都小于存储在 $v$ 中的元素，所有存储在顶点 $v$ 中的右子树中的元素都大于存储在 $v$ 中的元素。
- 用二叉搜索树来表示一个集合并不是唯一的，最坏的情况它可能是一棵退化的树，即这棵树的每一个内部顶点都恰好有一个儿子。
- 这种数据结构支持的操作有插入、删除、测试成员身份和检索最大或最小值。
- P72



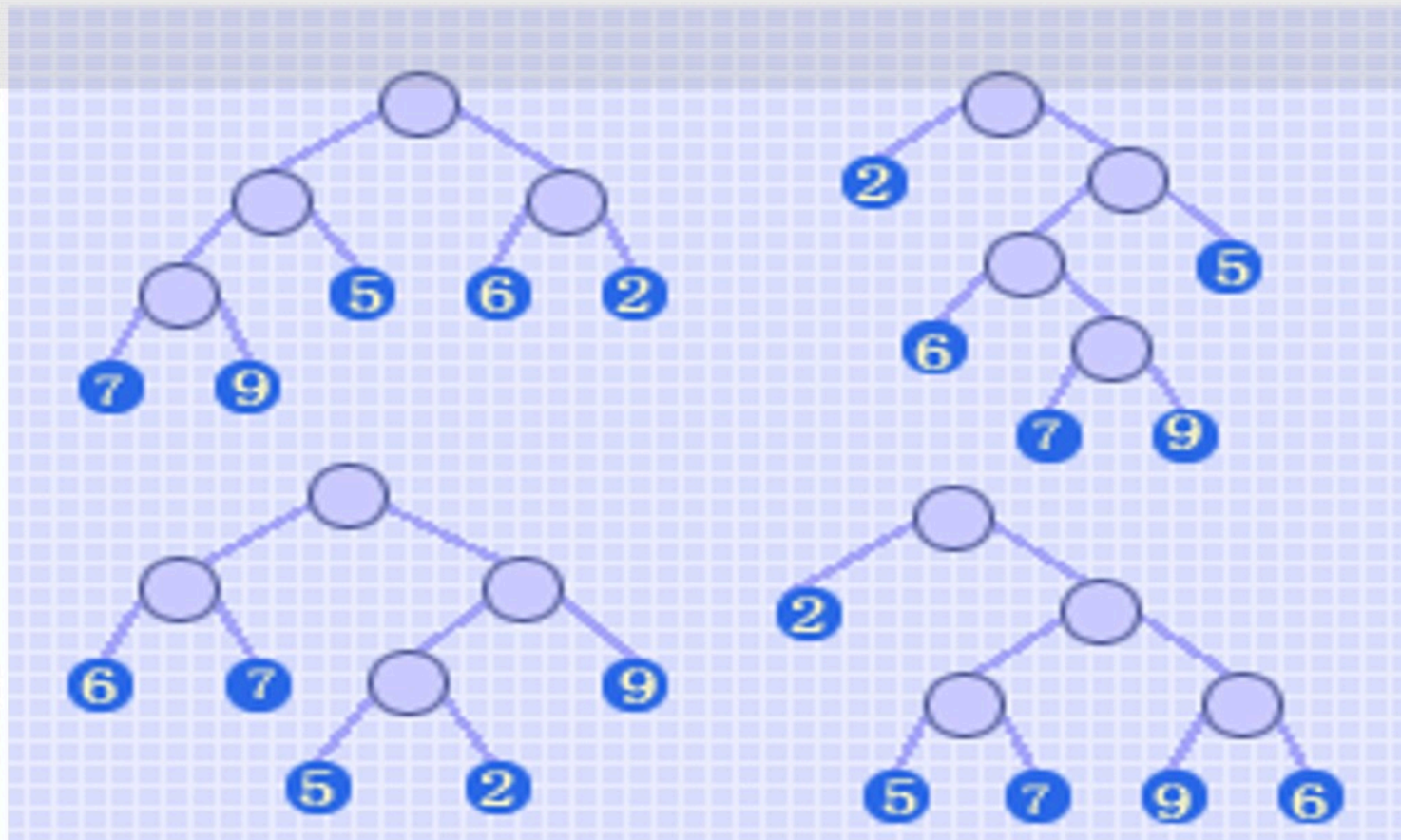


# 赫夫曼树及其应用

- **赫夫曼树**：带权路径长度最短的树称为赫夫曼树或最优树。
- **路径**：树的根结点到其余结点的分支。
- **结点的路径长度**：从根结点到该结点的路径上分支的数目。
- **结点的带权路径长度**：从树根到该结点之间的路径长度与该结点上所带权值的乘积。
- **树的带权路径长度**：树中所有叶子结点的带权路径长度之和。

$$WPL = \sum_{k=1}^n w_k l_k$$

- 在所有含 $n$ 个叶子结点、并带相同权值的 $m$ 叉树中，必存在一棵其带权路径长度取最小值的树，称为“**最优树**”



$$\begin{aligned}
 WPL &= 7 \times 3 + 9 \times 3 + 5 \times 2 + 6 \times 2 + 2 \times 2 = 74 \quad (\text{左上图}) \\
 WPL &= 2 \times 1 + 6 \times 3 + 7 \times 4 + 9 \times 4 + 5 \times 2 = 94 \quad (\text{右上图}) \\
 WPL &= 6 \times 2 + 7 \times 2 + 5 \times 3 + 2 \times 3 + 9 \times 2 = 65 \quad (\text{左下图}) \\
 WPL &= 2 \times 1 + 5 \times 3 + 7 \times 3 + 9 \times 3 + 6 \times 3 = 83 \quad (\text{右下图})
 \end{aligned}$$



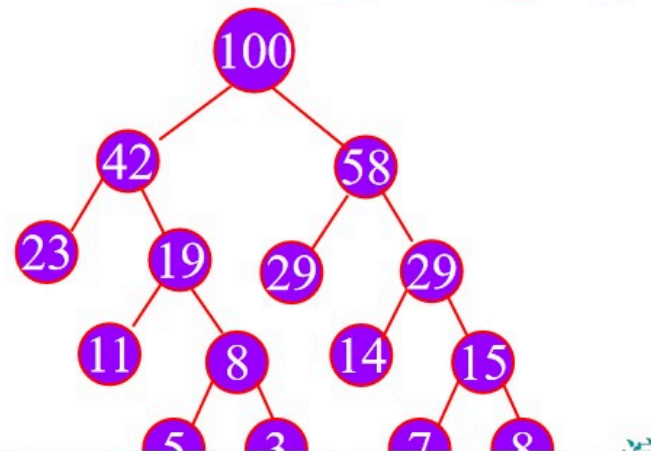
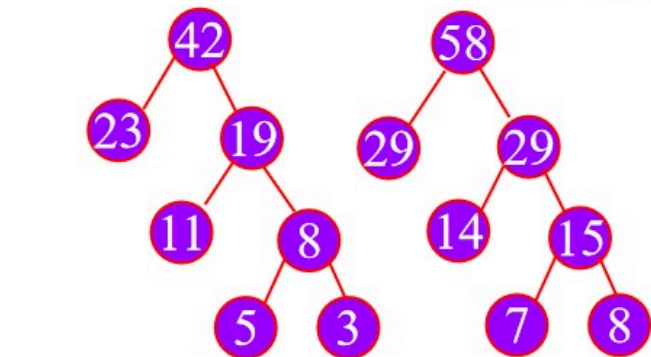
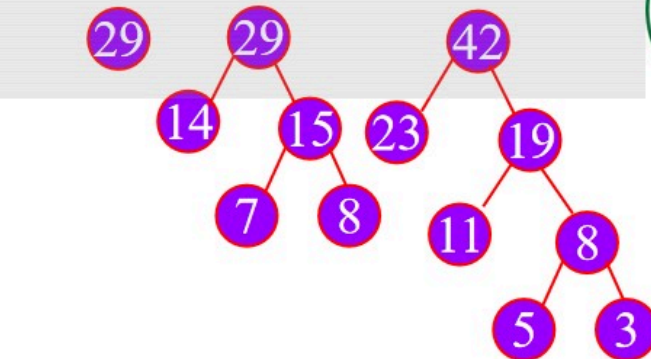
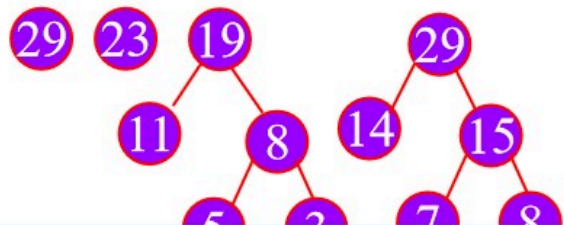
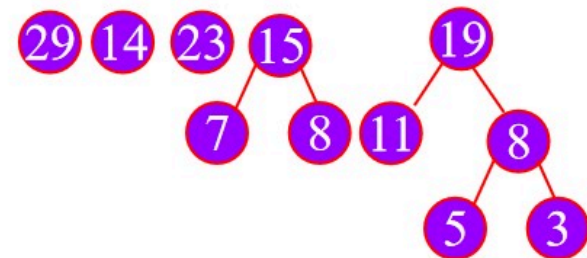
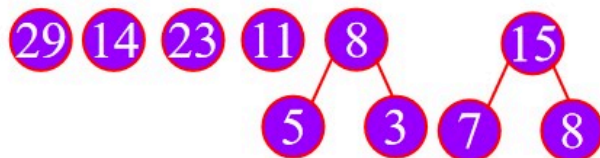


## 赫夫曼树的构造

- (1) 根据给定的 $n$ 个权值 $\{w_1, w_2, \dots, w_n\}$ , 构造 $n$ 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$ , 其中每棵二叉树中均只含一个带权值为 $w_i$ 的根结点, 其左、右子树为空树;
- (2) 在 $F$ 中选取其根结点的权值为最小的两棵二叉树, 分别作为左、右子树构造一棵新的二叉树, 并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和;  
(根结点的权值=左右孩子权值之和, 叶结点的权值= $w_i$ );
- (3) 从 $F$ 中删去这两棵树, 同时加入刚生成的新树;
- (4) 重复(2)和(3)两步, 直至 $F$ 中只含一棵树为止。

例  $w=\{5, 29, 7, 8, 14, 23, 3, 11\}$

5 29 7 8 14 23 3 11







# 赫夫曼树的应用（一）

- 1. 判定树

在很多问题的处理过程中，需要进行大量的条件判断，这些判断结构的设计直接影响着程序的执行效率。

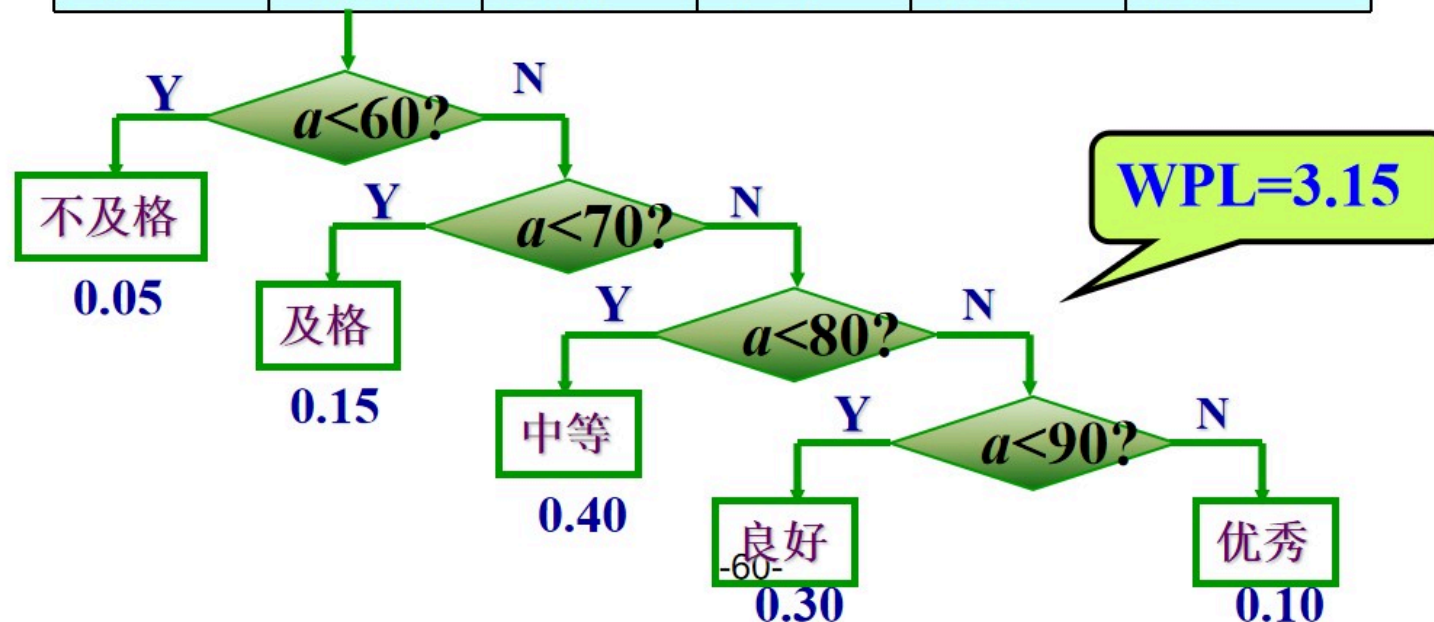
- 例如，编制一个程序，将百分制转换成五个等级输出。一种形式如下：

```
if (score < 60) printf("bad");
else if (score < 70) printf("pass");
else if (score < 80) printf("general");
else if (score < 90) printf("good");
else printf("very good");
```

# 赫夫曼树的应用（一）

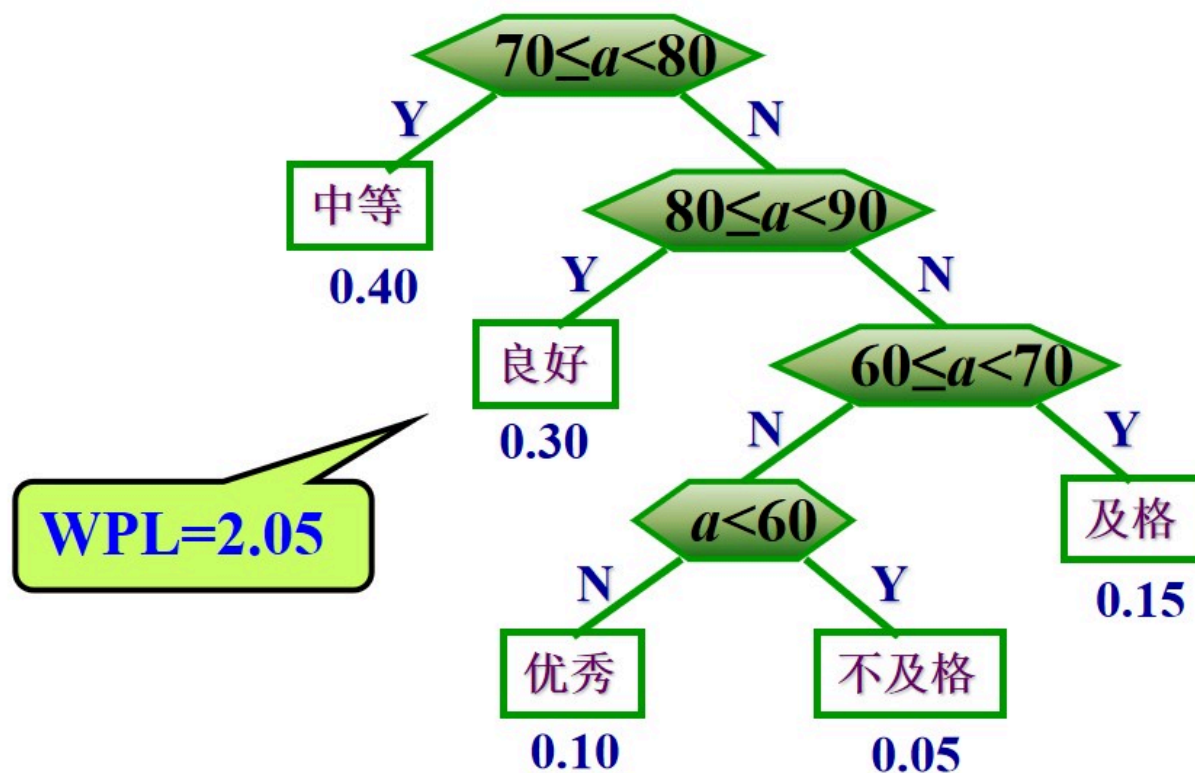
- 赫夫曼树的应用——最佳判定算法

| 分数 | 0~59 | 60~69 | 70~79 | 80~89 | 90~100 |
|----|------|-------|-------|-------|--------|
| 比例 | 0.05 | 0.15  | 0.40  | 0.30  | 0.10   |
| 等级 | 不及格  | 及格    | 中等    | 良好    | 优秀     |



# 赫夫曼树的应用（一）

- 赫夫曼树的应用——最佳判定算法







## 赫夫曼树的应用（二）

- 赫夫曼编码

在电文传输中，需要将电文中出现的每个字符进行二进制编码。在设计编码时需要遵守两个原则：

- (1) 发送方传输的二进制编码，到接收方解码后必须具有唯一性，即解码结果与发送方发送的电文完全一样；
- (2) 发送的二进制编码尽可能地短。





- 1. 等长编码

这种编码方式的特点是每个字符的编码长度相同（编码长度就是每个编码所含的二进制位数）。假设字符集只含有4个字符A, B, C, D, 用二进制两位表示的编码分别为00, 01, 10, 11。若现在有一段电文为: ABACCDA, 则应发送二进制序列:

00010010101100, 总长度为14位。当接收方接收到这段电文后, 将按两位一段进行译码。这种编码的特点是译码简单且具有唯一性, 但编码长度并不是最短的。

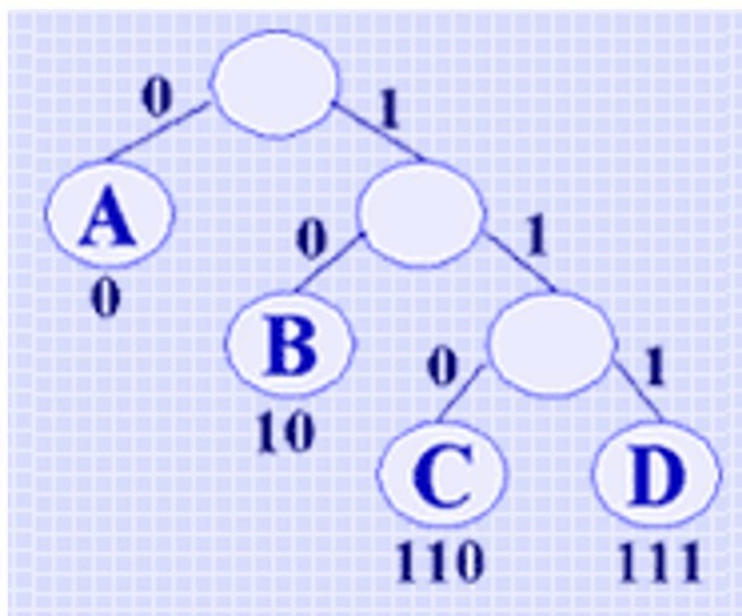
- 2. 不等长编码

在传送电文时, 为了使其二进制位数尽可能地少, 可以将每个字符的编码设计为不等长的, 使用频度较高的字符分配一个相对较短的编码, 使用频度较低的字符分配一个比较长的编码。例如, 可以为A, B, C, D四个字符分别分配0, 00, 1, 01, 并可将上述电文用二进制序列: 000011010发送, 其长度只有9个二进制位, 但随之带来了一个问题, 接收方接到这段电文后无法进行译码, 因为无法断定前面4个0是4个A, 1个B、2个A, 还是2个B, 即译码不唯一。

- 前缀编码

任何一个字符的编码都不是同一字符集中另一个字符的编码的前缀。

- 可以利用二叉树来设计二进制的前缀编码。用叶子结点表示字符，且约定左分支表示字符‘0’，右分支表示字符‘1’，则以由从根到叶子的路径上的分支表示的字符组成的字符串作为该叶子结点字符的编码。







## 赫夫曼树的应用（二）

- 在二叉树来表示的前缀编码中，**以字符出现的次数为权**，构造一棵赫夫曼树，由此得到的二进制前缀编码便为“最优前缀编码”（赫夫曼编码）。即以这组编码传送电文可使电文总长最短(对所有其它前缀编码而言)。
- 假设有一个电文字符集中有8个字符，每个字符的使用频率分别为 $\{0.05, 0.29, 0.07, 0.08, 0.14, 0.23, 0.03, 0.11\}$ ，以此为例设计赫夫曼编码。



## 赫夫曼树的应用（二）

- 赫夫曼编码设计过程为：

(1) 为方便计算，将所有字符的频度乘以100，使其转换成整型数值集合，得到{5,29,7,8,14,23,3,11}；

(2) 以此集合中的数值作为叶子结点的权值构造一棵赫夫曼树

(3) 由此赫夫曼树生成赫夫曼编码

| 字符 | 字符的使用频率 | 编码   |
|----|---------|------|
| ①  | 0.05    | 0110 |
| ②  | 0.29    | 10   |
| ③  | 0.07    | 1110 |
| ④  | 0.08    | 1111 |
| ⑤  | 0.14    | 110  |
| ⑥  | 0.23    | 00   |
| ⑦  | 0.03    | 0111 |
| ⑧  | 0.11    | 010  |

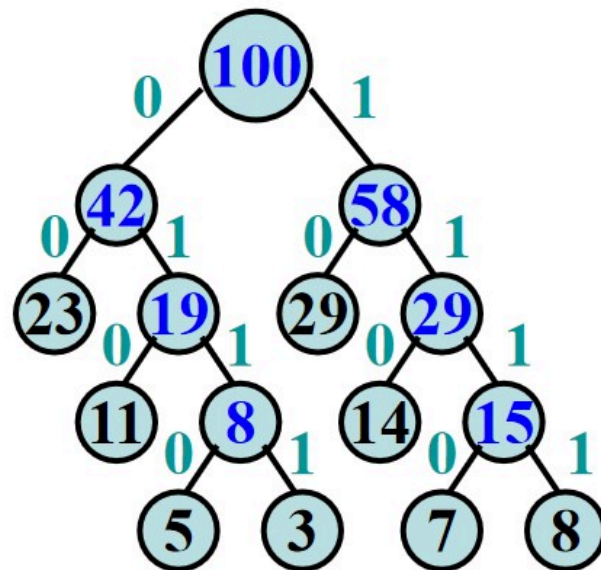
以上的①~⑧分别代表 8 个字符

华中农业大学信息学院



## 赫夫曼树的应用（二）

- 赫夫曼编码的算法



平均码长

$$\sum_{i=1}^8 p_i l_i = (0.23+0.29)*2 + (0.11+0.14)*3 + (0.05+0.03+0.07+0.08)*4$$

$$= 1.04 + 0.75 + 0.92$$

$$= 2.71$$

若电文有1000个字符，则电文的码长为  $2.71 * 1000 = 2710$



### 3.3 图

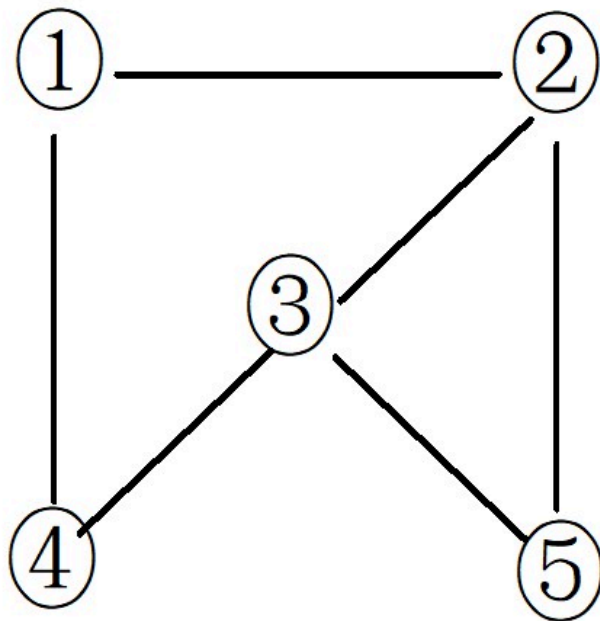
- 图是一种比线性表和树更为复杂的数据结构。在图中，元素间的关系是多对多的，即任何两个元素都有可能存在关系。
- 图的应用非常广泛，已渗入到诸如语言学、逻辑学、物理、化学、电讯工程、计算机科学以及数学的其它分支中(日常生活中的交通图等)。
- 在离散数学中，图论是专门研究图的性质的数学分支。
- 在数据结构中对图的讨论主要侧重于图在计算机中的存储方式和有关操作的算法。



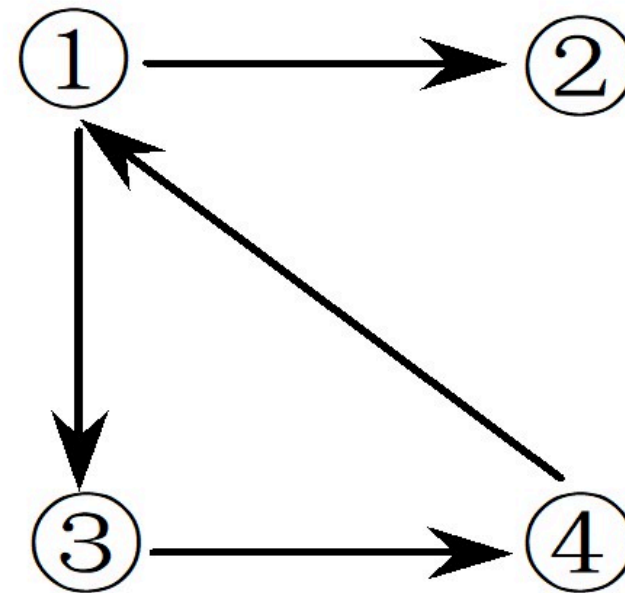
## 3.3 图

- 图:
  - 图  $G=(V,E)$  由一个顶点集合  $V=\{v_1, v_2, \dots, v_n\}$  和一个边的集合  $E$  组成。  $G$  可以是有向的也可以是无向的。如果  $G$  是**无向的**，那么  $E$  中的每一条边都是无序的顶点对；如果  $G$  是**有向的**，那么  $E$  中的每一条边都是一个有序的顶点对。
- 邻接:
  - 设  $(v_i, v_j)$  是  $E$  的一条边，如果图是无向的，那么  $v_i$  和  $v_j$  互相邻接，如果图是有向的，那么  $v_j$  邻接于  $v_i$ ，但  $v_i$  不邻接于  $v_j$ ，除非  $(v_j, v_i)$  也是  $E$  中一条边。

## 图的示例



无向图 $G_1$



有向图 $G_2$





## 图

- 度:
  - 无向图中顶点的度是和它邻接的顶点的个数，有向图中顶点  $v_i$  的入度和出度分别是连接到  $v_i$  的边和从  $v_i$  连接到其他顶点的边的数目。
- 路径:
  - 图中从顶点  $v_1$  到  $v_k$  的路径是一个顶点的序列  $v_1, v_2, \dots, v_k$ ，其中的  $(v_i, v_{i+1}), 1 \leq i \leq k-1$ ，是图的一条边。
  - 路径的长度是路径中边的数目，因此，路径  $v_1, v_2, \dots, v_k$  的长度是  $k-1$ 。如果所有的顶点都不同，那么路径是简单的；如果  $v_1 = v_k$ ，那么路径是一个回路。
  - 奇数长度的回路，其边的数目是奇数，类似的方法可以定义偶数长度的回路。
  - 没有回路的图称为无回路图。如果有一条路径从顶点  $u$  开始到顶点  $v$  结束，那么称顶点  $v$  从  $u$  出发是可达的。



## 图

- **连通的:**

- 如果每个顶点从其他每个顶点出发都是可达的，那么这个无向图是连通的，否则就是不连通的。一个图的连通分支是图的**最大连通子图**。
- 在有向图中，如果子图中每一对顶点 $u$ 和 $v$ 满足 $v$ 是从 $u$ 出发可达的，同时， $u$ 也是从 $v$ 出发可达的，这个子图就称为**强连通分支**。



## 图

- **完全图:**

- 如果一个无向图的每一对顶点之间都恰有一条边, 那么这个无向图就称为**完全图**。如果一个有向图的每个顶点到所有其他顶点之间都恰有一条边, 那么这个有向图称为完全图。

- **二分图:**

- 如果在无向图 $G=(V,E)$ 中,  $V$ 可以分成两个不相交的子集 $X$ 和 $Y$ , 使得 $E$ 中的每一条边的一端在 $X$ 中而另一端在 $Y$ 中, 则无向图称为二分图。如果在任意的顶点 $x \in X$ 和 $y \in Y$ 之间都有一条边, 它就称为**完全二分图**。





## 3.3.1 图的表示

- 邻接矩阵:

- 图 $G=(V,E)$  可以用一个布尔矩阵 $M$ 方便的表示,它是这样定义的,当且仅当  $(v_i,v_j)$  是 $G$ 中的一条边时,  $M[i,j]=1$ , 矩阵 $M$ 称为 $G$ 的邻接矩阵。有 $n$ 个顶点的图的邻接矩阵有 $n^2$ 项。

- 邻接表:

- 图的另一种表示法是邻接表表示法。一个顶点的所有邻接顶点用一个链表来表示,共有  $|V|$  个这样的表。邻接表要花费  $\Theta(m+n)$  控件来表示有  $n$  个顶点和 $m$ 条边的图。



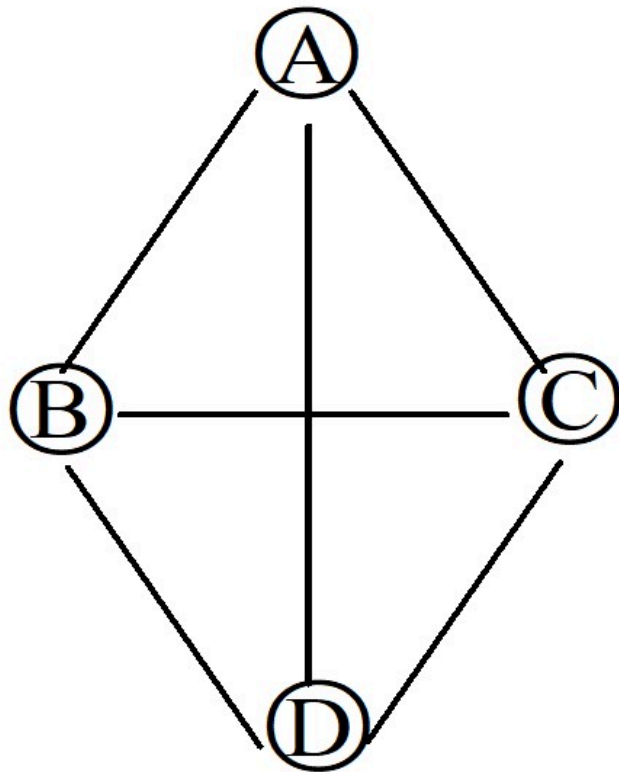


## 邻接矩阵

- **邻接矩阵**：表示顶点之间相邻关系的矩阵。所谓两顶点的相邻关系即它们之间有边相连。
- **邻接矩阵**是一个  $(n \times n)$  阶方阵， $n$  为图的顶点数，它的每一行分别对应图的各个顶点，它的每一列也分别对应图的各个顶点。我们规定矩阵的元素为：

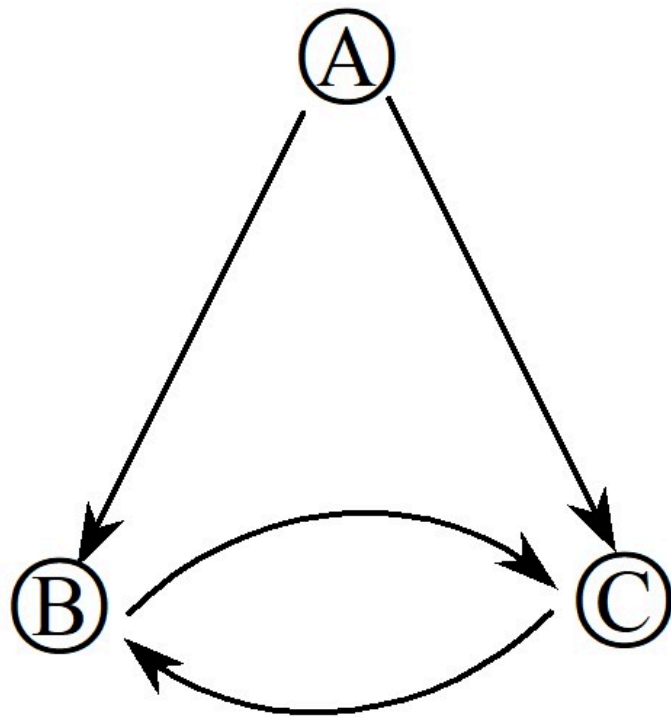
$$A[i, j] = \begin{cases} 1 & \text{对无向图若存在}(V_i, V_j)\text{边，对有向图若存在}\langle V_i, V_j \rangle\text{边} \\ 0 & \text{反之} \end{cases}$$

## 无向图的邻接矩阵



$$A = \begin{matrix} & \begin{matrix} A & B & C & D \end{matrix} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} & \begin{matrix} A \\ B \\ C \\ D \end{matrix} \end{matrix}$$

## 有向图的邻接矩阵



$$B = \begin{matrix} & \begin{matrix} A & B & C \end{matrix} \\ \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} & \begin{matrix} A \\ B \\ C \end{matrix} \end{matrix}$$



## 邻接矩阵的若干说明

- 无向图的邻接矩阵是**对称的**，如果 $A[i, j]=1$ ，必有 $A[j, i]=1$ 。这说明，只输入和存储其上三角阵元素即可得到整个邻接矩阵。
- 一般有向图的邻接矩阵是不对称的， $A[i, j]$ 不一定等于 $A[j, i]$ 。
- 邻接矩阵用二维数组即可存储，定义如下：  

```
int adjmatrix = ARRAY[n][n];
```
- 如果图的各边是带权的，只需将矩阵中的各个1元素换成相应边的权即可。

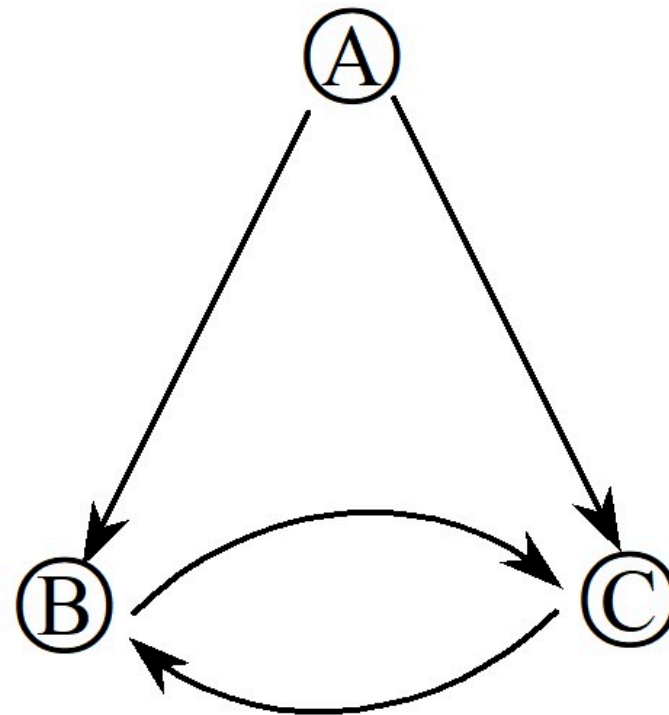
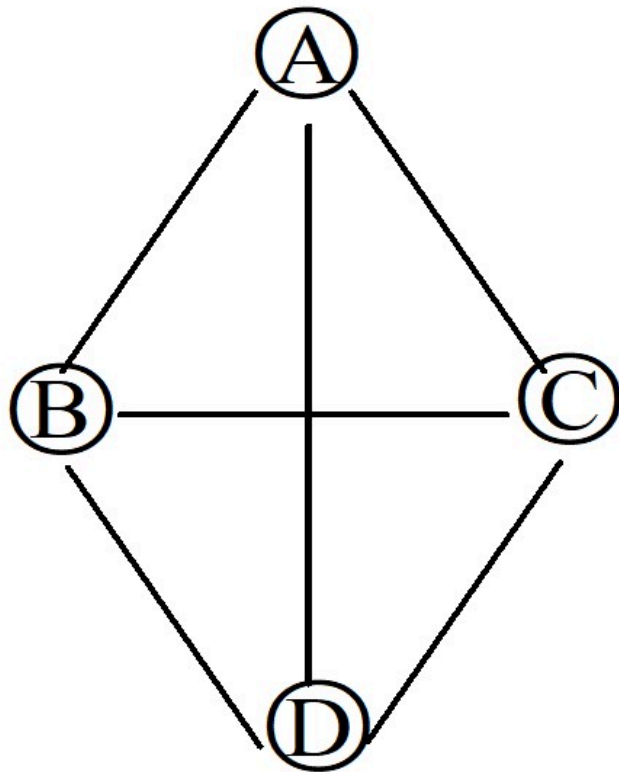




# 邻接表

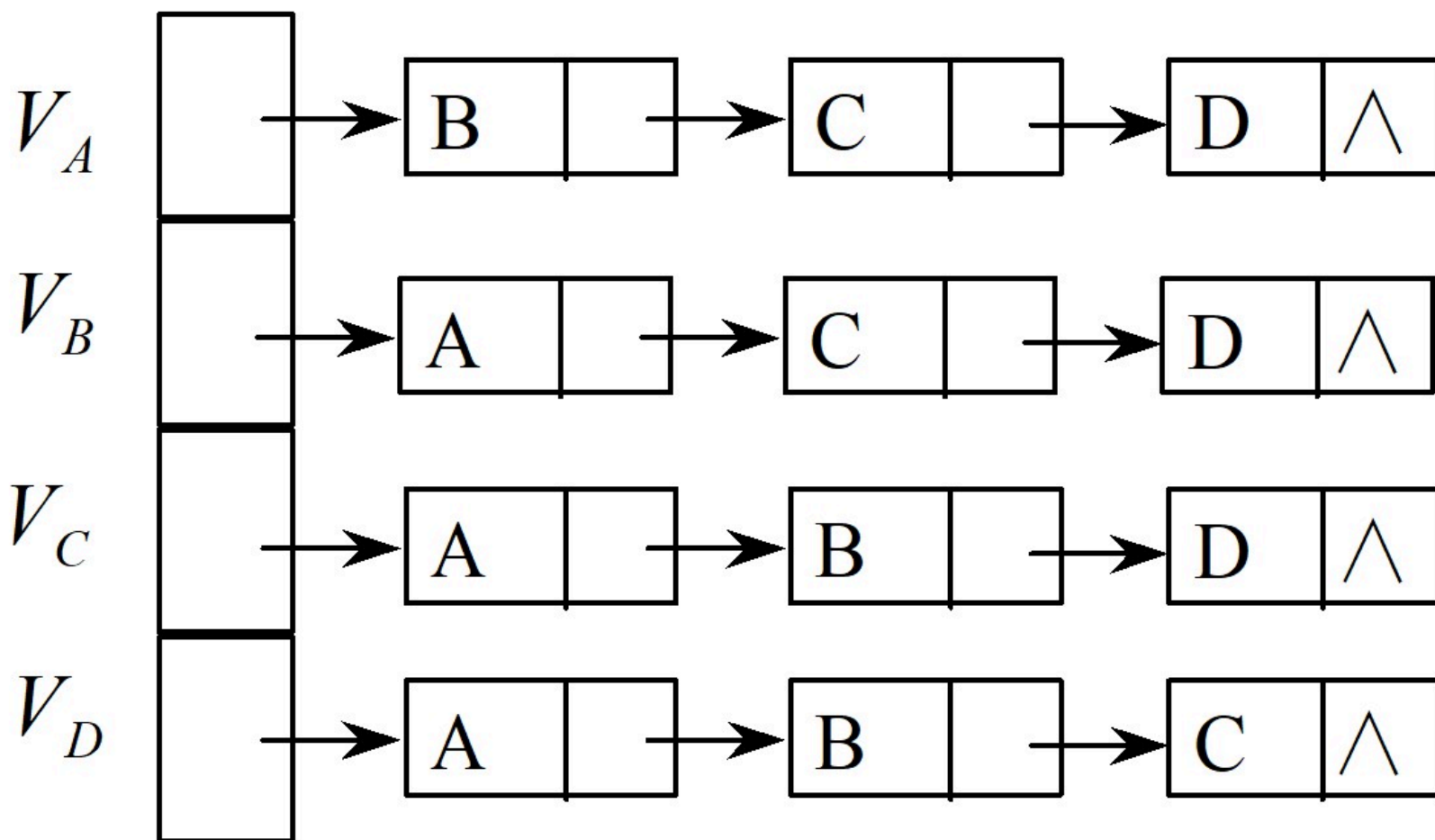
- 邻接表是图的一种链式存储结构。
- 在邻接表结构中，对图中每个顶点建立一个单链表，第 $i$ 个单链表中的结点表示依附于该顶点 $V_i$ 的边，即对于无向图每个结点表示与该顶点相邻接的一个顶点；对于有向图则表示以该顶点为起点的一条边的终点。
- 一个图的邻接矩阵表示是唯一的，但其邻接表表示是不唯一的。因为在邻接表的每个单链表中，各结点的顺序是任意的。

## 无向图、有向图



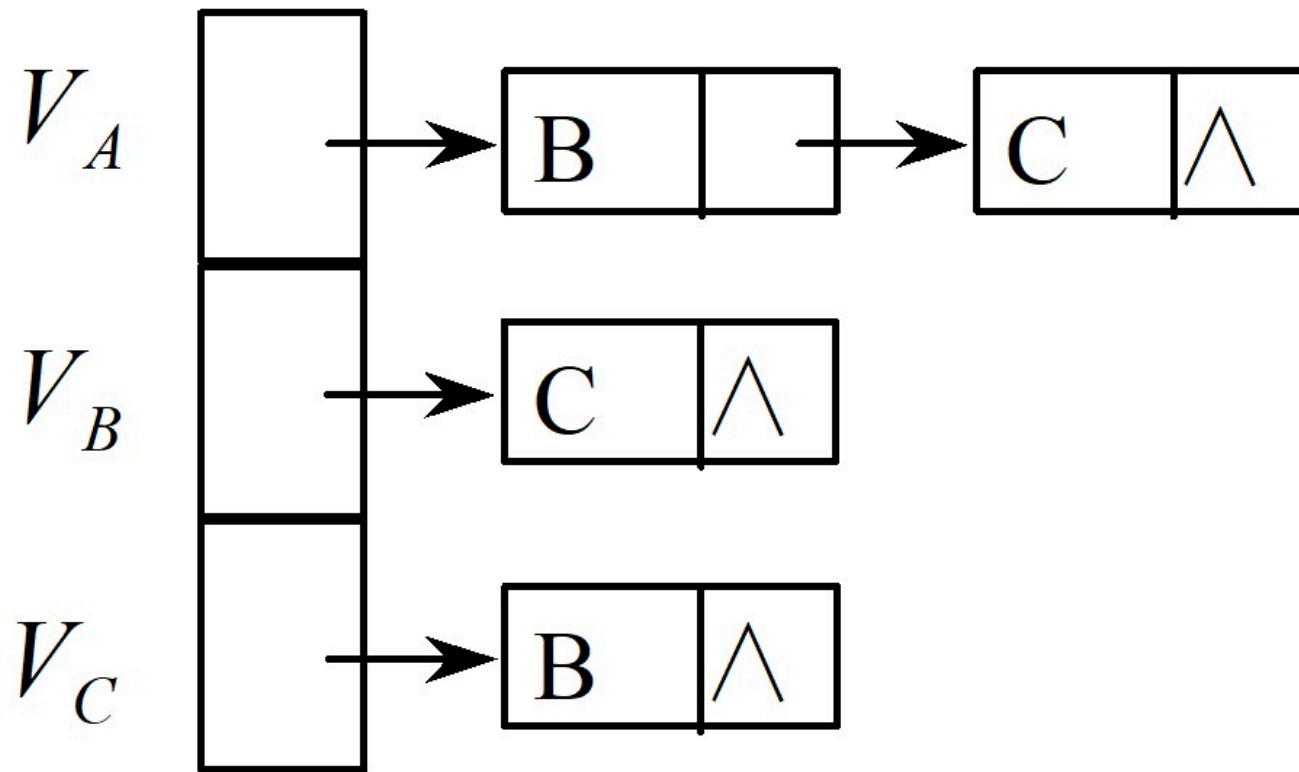


# 无向图的邻接表





## 有向图的邻接表





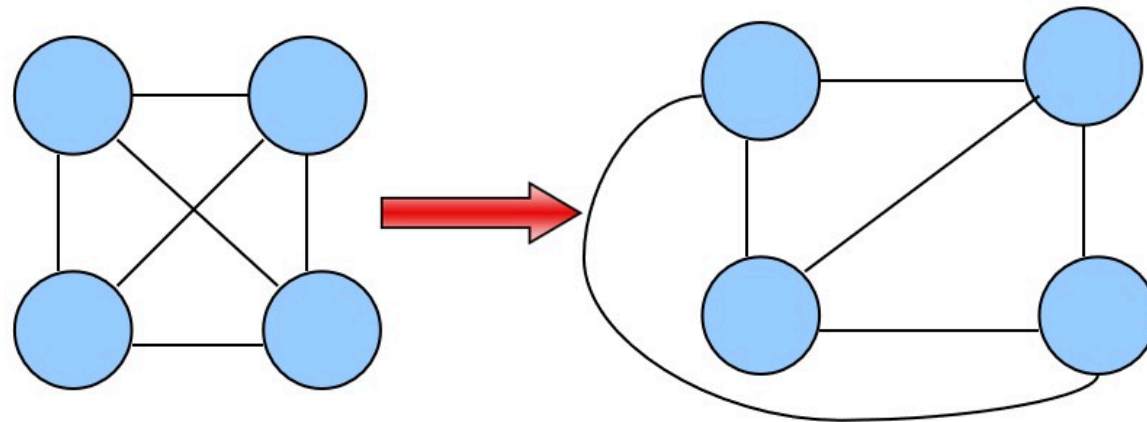


## 邻接表的若干说明

- 对于无向图的邻接表来说，一条边对应两个单链表结点，邻接表结点总数是边数的2倍。
- 在无向图的邻接表中，各顶点对应的单链表的结点数（不算表头结点）就等于该顶点的度数。
- 在有向图邻接表中，一条弧对应一个表结点，表结点的数目和弧的数目相同。
- 在有向图邻接表中，单链表的结点数就等于相应顶点的出度数。
- 要求有向图中某顶点的入度数，需扫视邻接表的所有单链表，统计与顶点标号相应的结点个数。

### 3.3.2 平面图

- 如果图 $G=(V,E)$  可以嵌入平面而没有任何边互相穿越，这个图就是平面图。





## 7.6 最短路径（图）

- ❖ 我们研究带权图，一个重要的内容就是寻找某类**具有最小（最大）权的子图**，其中之一就是最短路问题，例如：给定一个连接各城市的铁路网络（连通的带权图），在这个网络中的两个指定的城市之间确定一条最短路。
- ❖ 所谓**最短路径**（shortest path）问题指的是：如果从图中某顶点出发（此点称为源点），经图的边到达另一顶点（称为终点）的路径不止一条，如何找到一条路径使沿此路径上各边的权值之和为最小。
- ❖ 设一有向网络 $G = (V, E)$ ，已知各边的权值，并设每边的权均大于零，以某指定 $V_0$ 为源点，求从 $V_0$ 到图的其余各点的最短路径。





求最短路长的算法是E.W.Dijkstra（狄克斯特拉）于1959年提出来的，这是至今公认的求最短路长的最好算法，我们称它为Dijkstra算法。

- Dijkstra算法

功能：在连通的带权图中，求从 $v_0$ 到 $v$ 的最短路的路长。

No1.  $p_0 = v_0$ ;  $P = \{v_0\}$ ;  $T = V \setminus \{v_0\}$ ;  $d(p_0) = 0$ ;

$(\forall t \in T)(d(t) = \infty)$ ;

No2.  $(\forall t \in T)(d(t) = \min(d(t), d(p_0) + W(p_0, t)))$ ;

No3. 在 $T$ 中选取 $t_0$ , 使 $(\forall t \in T)(d(t_0) \leq d(t))$ ;

No4.  $p_0 = t_0$ ;  $P = P \cup \{t_0\}$ ;  $T = T \setminus \{t_0\}$ ;

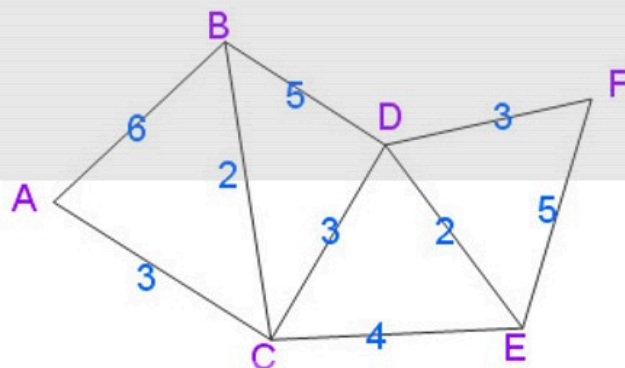
No5. if  $p_0 \neq v$  then goto No2 else end.





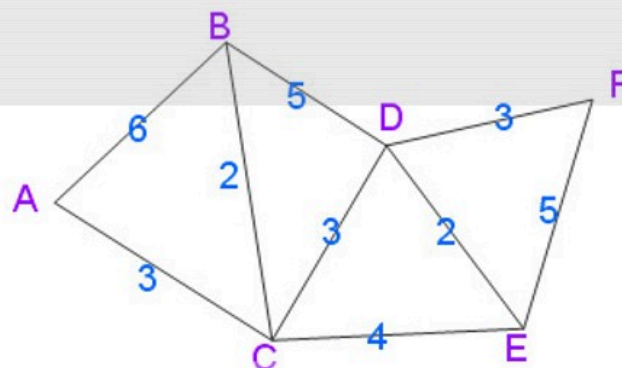
- Dijkstra算法步骤

- (1) 初始时,  $S$  只包含源点, 即  $S=v$  的距离为0。  $U$  包含除  $v$  外的其他顶点,  $v$  到  $U$  中顶点  $u$  的距离为  $v$  到  $u$  边上的权。
- (2) 从  $U$  中选取一个距离  $v$  最小的顶点  $k$ , 把  $k$  加入  $S$  中 (该选定的距离就是  $v$  到  $k$  的最短路径长度)。
- (3) 以  $k$  为新考虑的中间点, 修改  $U$  中各顶点的距离; 若从源点  $v$  到顶点  $u$  ( $u \in U$ ) 的距离 (经过顶点  $k$ ) 比原来距离 (不经过顶点  $k$ ) 短, 则修改顶点  $u$  的距离值。
- (4) 重复步骤 (2) 和 (3) 直到所有顶点都包含在  $S$  中。



| 步骤 | S 集合中                                                                                                                                                                                       | U 集合中                                                                                                                                                                                                                                                                                                                                                         |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 选入 A, 此时 $S = \langle A \rangle$<br>此时最短路径 $A \rightarrow A = 0$<br>以 A 为中间点, 从 A 开始找                                                                                                       | $U = \langle B, C, D, E, F \rangle$<br>$A \rightarrow B = 6$<br>$A \rightarrow C = 3$<br>$A \rightarrow \text{其他 } U \text{ 中的顶点} = \infty$<br>发现 $A \rightarrow C = 3$ 权值为最短                                                                                                                                                                                 |
| 2  | 选入 C, 此时 $S = \langle A, C \rangle$<br>此时最短路径 $A \rightarrow A = 0, A \rightarrow C = 3$<br>以 C 为中间点, 从 $A \rightarrow C = 3$ 这条最短路径开始找                                                     | $U = \langle B, D, E, F \rangle$<br>$A \rightarrow C \rightarrow B = 5$ (比上面第一步的 $A \rightarrow B = 6$ 要短)<br>此时到 B 权值为 $A \rightarrow C \rightarrow B = 5$<br>$A \rightarrow C \rightarrow D = 6$<br>$A \rightarrow C \rightarrow E = 7$<br>$A \rightarrow C \rightarrow \text{其他 } U \text{ 中的顶点} = \infty$<br>发现 $A \rightarrow C \rightarrow B = 5$ 权值为最短 |
| 3  | 选入 B, 此时 $S = \langle A, C, B \rangle$<br>此时最短路径 $A \rightarrow A = 0, A \rightarrow C = 3, A \rightarrow C \rightarrow B = 5$<br>以 B 为中间点, 从 $A \rightarrow C \rightarrow B = 5$ 这条最短路径开始找 | $U = \langle D, E, F \rangle$<br>$A \rightarrow C \rightarrow B \rightarrow D = 10$ (比上面第二步的 $A \rightarrow C \rightarrow D = 6$ 要长)<br>此时到 D 权值更改为 $A \rightarrow C \rightarrow D = 6$<br>$A \rightarrow C \rightarrow B \rightarrow \text{其他 } U \text{ 中的顶点} = \infty$<br>发现 $A \rightarrow C \rightarrow D = 6$ 权值为最短                                     |

华中农业大学信息学院



|   |                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                   |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | <p>选入 D, 此时 <math>S = \langle A, C, B, D \rangle</math></p> <p>此时最短路径 <math>A \rightarrow A = 0</math>, <math>A \rightarrow C = 3</math>,<br/> <math>A \rightarrow C \rightarrow B = 5</math>, <math>A \rightarrow C \rightarrow D = 6</math></p> <p>以 D 为中间点, 从 <math>A \rightarrow C \rightarrow D</math> 这条最短路径开始找</p>                                                            | <p><math>U = \langle E, F \rangle</math></p> <p><math>A \rightarrow C \rightarrow D \rightarrow E = 8</math> (比上面第二步的 <math>A \rightarrow C \rightarrow E = 7</math> 要长)</p> <p>此时到 E 权值更改为 <math>A \rightarrow C \rightarrow E = 7</math></p> <p><math>A \rightarrow C \rightarrow D \rightarrow F = 9</math></p> <p>发现 <math>A \rightarrow C \rightarrow E = 7</math> 权值为最短</p> |
| 5 | <p>选入 E, 此时 <math>S = \langle A, C, B, D, E \rangle</math></p> <p>此时最短路径 <math>A \rightarrow A = 0</math>, <math>A \rightarrow C = 3</math>,<br/> <math>A \rightarrow C \rightarrow B = 5</math>, <math>A \rightarrow C \rightarrow D = 6</math><br/> <math>A \rightarrow C \rightarrow E = 7</math></p> <p>以 E 为中间点, 从 <math>A \rightarrow C \rightarrow E = 7</math> 这条最短路径开始找</p> | <p><math>U = \langle F \rangle</math></p> <p><math>A \rightarrow C \rightarrow E \rightarrow F = 12</math> (比上面第四步的 <math>A \rightarrow C \rightarrow D \rightarrow F = 9</math> 要长) 此时到 F 权值更改为 <math>A \rightarrow C \rightarrow D \rightarrow F = 9</math></p> <p>发现 <math>A \rightarrow C \rightarrow D \rightarrow F = 9</math> 权值为最短</p>                                    |
| 6 | <p>选入 F, 此时 <math>S = \langle A, C, B, D, E, F \rangle</math></p> <p>此时最短路径 <math>A \rightarrow A = 0</math>, <math>A \rightarrow C = 3</math>,<br/> <math>A \rightarrow C \rightarrow B = 5</math>, <math>A \rightarrow C \rightarrow D = 6</math><br/> <math>A \rightarrow C \rightarrow E = 7</math>, <math>A \rightarrow C \rightarrow D \rightarrow F = 9</math></p>            | <p>U 集合已空, 查找完毕。</p>                                                                                                                                                                                                                                                                                                                                                              |

2728areen-rock.blog.163.com





- Dijkstra算法的基本思想是：将图  $G$  中结点集合  $V$  分成两部分，一部分称为具有  $P$  标号的集合，另一部分称为具有  $T$  标号的集合。
- 所谓结点  $a$  的  $P$  标号是指从  $v_0$  到  $a$  的最短路的路长，而结点  $b$  的  $T$  标号是指从  $v_0$  到  $b$  的某条路径的长度。
- Dijkstra算法中首先将  $v_0$  取为  $P$  标号结点，其余的点均为  $T$  标号结点，然后逐步地将具有  $T$  标号的结点改为  $P$  标号结点，当结点  $v$  也被改为  $P$  标号时，就找到了从  $v_0$  到  $v$  的最短路径的长度。